Cloud Computing

Exam Revision Notes – Weeks 1 to 12 (MCQ / MSQ focus)
Based on Prof. Soumya K. Ghosh's NPTEL lectures, IIT Kharagpur

W1 Cloud Computing Overview, Architecture & Service Models
W2 Deployment Models, Virtualization, XML, Web Services & SOA
W3 SLA, Cloud Economics, Data Management (GFS/BigTable/Dynamo), MapReduce Intro & OpenStack
W4 Private Cloud on OpenStack (Meghamala), Azure Web App, Google Cloud Platform
W5 SLA / Economics / MapReduce Tutorials, Resource Management I & II
W6 Cloud Security I–IV & Broker for Cloud Marketplace
W7 Mobile Cloud Computing, Fog Computing, Geospatial Cloud
W8 Docker Containers, Green Cloud, Sensor Cloud, IoT Cloud, Course Summary
W9 Cloud–Fog–Edge Paradigm, Resource Management & Service Placement, Cloud Federation
W10 VM Migration (Live Migration) & Containers (Docker, Kubernetes)
W11 Dew Computing, Serverless Computing (FaaS), Sustainable Cloud Computing
W12 Cloud in 5G Era, Cyber-Physical Systems, Spatial Cloud, Internet of Health Things (IoHT) Case Study

WEEK 1

Cloud Computing Overview, Architecture & Service Models

1.1 Trends in Computing (evolution order)

- Order: **Distributed → Grid → Cluster → Utility → Cloud** computing.
- Early computing = single processor = **centralized (uni-processor)** computing.

Distributed Computing

- Distributed system: several **autonomous** computational entities, each with **own local memory**, communicating by **message passing**.
- Examples: Internet, ATM (bank) machines, Intranets/workgroups.
- Computers in a DS: **Workstations** (end users), **Server systems** (provide resources/services), **Personal assistance devices** (handhelds via wireless).
- Common properties: **fault tolerance**, each node plays a partial role (limited/incomplete view), **resource sharing**, **load sharing**, easy to expand, performance.
- **Parallel computing is a subset of distributed computing.**
- Why distributed: nature of application, **computing-intensive** (e.g., Pi by Monte Carlo), **data-intensive** (e.g., Facebook, LHC data), **robustness – no SPOF** (Single Point of Failure).
- Past distributed apps were mostly **client-server** (centralized resource management at server); **Peer-to-Peer (P2P)** = more "truly" distributed.

Grid Computing

- Harnesses **unused processing cycles** of all computers in a network to solve problems too big for one machine.
- Gives a **single system image / one large virtual computer**; users don't know where resources are.
- **Electrical power grid analogy**: wall socket = access to computing; grid links computing resources like power grid links power plants.
- Used for: sharing data/compute/apps in **virtual organisations (VO)**, multi-institutional collaboration.
- **Types of grids**: **Computational** (processing power), **Data** (storage/discovery/handling of data), **Collaboration** (virtual enterprise work, e.g., CAD without disclosing proprietary tech), **Network** (fault-tolerant comms; node works as data router/cache), **Utility** (ultimate form; shares software/special equipment as well).

Cluster Computing

- Collection of **interconnected stand-alone computers** working together as a **single integrated resource**.
- Components: standalone computers (PCs/workstations/SMPs), OS, **high-performance interconnects**, **middleware**, parallel programming environments, applications.
- Faster, closer connection than LAN; **low-latency protocols**; **loosely coupled compared to SMP**.
- Types: **High Availability (Failover)**, **Load Balancing**, **Parallel/Distributed Processing** clusters.
- Benefits: system availability, hardware fault tolerance (e.g., RAID), OS/app reliability, **scalability**, high performance.

Utility Computing

- A **concept** that cloud practically implements; resources **rented**, billed by **usage not flat rate (pay-per-use)**.
- Low/no initial cost. Analogy: electricity, water, gas.
- Features: pay-for-use pricing, data-center virtualization & provisioning, solves resource-utilization problem, outsourcing, web services delivery, automation.
- Payment models: flat rate, tiered, subscription, **metered**, pay-as-you-go, standing charges.
- Risks: data backup, data security, partner competency, defining SLA, charge-back value.

1.2 Cloud Computing – NIST Definition

- NIST: model for **ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources** (networks, servers, storage, applications, services), **rapidly provisioned and released with minimal management effort**.

5 Essential Characteristics (NIST)

- **On-demand self-service** – consumer provisions automatically without human interaction with provider.
- **Broad network access** – via standard mechanisms; thin/thick clients (phones, tablets, laptops).
- **Resource pooling** – **multi-tenant model**; physical/virtual resources dynamically assigned/reassigned.
- **Rapid elasticity** – scale outward/inward; appears unlimited to consumer.
- **Measured service** – metering; usage monitored, controlled, reported (transparency).

Common Characteristics

- Massive scale, resilient computing, homogeneity, geographic distribution, **virtualization**, service orientation, low-cost software, advanced security.

Service Models (NIST)

- **SaaS**: use provider's applications via thin client/browser; consumer controls nothing except limited user config. e.g., Google Spreadsheet, web email, Salesforce CRM.
- **PaaS**: deploy consumer-created apps using provider-supported languages/tools; controls **apps + hosting config**, not infra. e.g., **Microsoft Azure, Google App Engine**.
- **IaaS**: provision processing, storage, networks; consumer can deploy **arbitrary software incl. OS**. e.g., **Amazon AWS, Flexiscale**.

Deployment Models

- **Private** – operated solely for one organisation (e.g., Windows Server Hyper-V).
- **Community** – shared by several organisations with a common goal.
- **Public** – available to general public (e.g., Google Docs).
- **Hybrid** – two or more clouds; e.g., **cloud bursting** for load balancing.

Virtualization

- **Hypervisor / VMM** sits between hardware and VMs; intercepts/emulates instructions; examples **VMware, Xen, KVM**.
- **Para-virtualization (Xen)** performance close to raw physical.
- Advantages: run OS where hardware unavailable, easy create/backup/migrate, testing on clean installs, emulate more machines than physical, timeshare lightly loaded systems, run legacy systems.

1.3 Advantages & Disadvantages of Cloud

Advantages

- Lower computer costs, improved performance, reduced software costs, **instant software updates**, improved document format compatibility, **unlimited storage**, **increased data reliability**, universal information access, latest version availability, easier group collaboration, device independence.

Disadvantages

- Needs **constant internet**; poor with low-speed connections; features may be limited; can be slow; **data security** concerns; data can be lost (no local backup).
- **HPC (MPI/OpenMP) apps** are a problem – need co-located VMs for low latency.
- **No standard API** (mix of SOAP and REST); different clouds use different protocols → **vendor lock-in/interoperability** issues.
- Cloud-sourcing concerns: performance/reliability/SLAs, control of data, privacy, security, compliance, trust.

1.4 Business Drivers & Economics

- Private data centres have low **average utilization of 5–20%** (provision for peak).
- Cloud gives **illusion of infinite capacity**, pay only for consumed resource.
- IaaS economics example (Shroff): In-house $0.36/hr vs Cloud $0.68/hr (ratio 1.88); efficiency 40% vs 80%; **total cost/effective hr $1.36 vs $0.86 → in-house/cloud = 1.58**.
- Public cloud benefits: high utilization, scalability, no hardware procurement/power cost/admin, SLA ~**99% availability** with compensation on SLA miss, prototype testing.
- Portability bodies: **Cloud Computing Interoperability Forum, Open Cloud Consortium**; **Amazon EC2 and Eucalyptus share same API**.
- Private cloud: 1 server with 12 cores/12GB cheaper than 12 servers of 1 core/1GB; confidentiality preserved; VMs cheaper and faster to provision.
- **PaaS vs IaaS**: for unpredictable 24×7 web app, IaaS needs minimal servers always on; PaaS costs nothing until used and **auto-scales**.
- PaaS benefits: no scaling/load-balancing handling, web-based IDE, easy dev→production migration.
- SaaS benefits: subscribe instead of buying licences; e.g., CRM without own servers.

1.5 Cloud Architecture & XaaS

- **XaaS = "anything/everything as a service"**; essence of cloud; = **Cloud (broad network access + resource pooling) + SOI (on-demand business-driven infra + service orientation)**.
- Other XaaS: BPaaS, Storage aaS, Security (SECaaS), Database (DaaS), Monitoring/Management (MaaS), CaaS, Identity (IDaaS), Backup (BaaS), Desktop (DaaS).
- CSP requirements: increase productivity, end-user satisfaction, innovation, agility.
- **Legacy IT vs Evergreen IT**: Legacy = all layers managed by user, tightly-coupled vertical stack, manual provisioning.

- Technical architecture: XaaS stack, middleware, management & security. Deployment architecture: **geo-location check (legal/export)**, operation & monitoring.

Client-Server vs Cloud model

- Client-server: simple service model; may/may not be load balanced; limited scalability; **no virtualization**.
- Cloud: variety of service models (IaaS/PaaS/SaaS); load balanced; theoretically infinitely scalable; **virtualization is core**.
- 3-tier client-server: presentation, application, data tiers.

Who manages what (XaaS stack)

- Stack layers: Applications, Data, Runtime, Middleware, O/S, Virtualization, Servers, Storage, Network.
- **IaaS**: user manages Apps, Data, Runtime, Middleware, O/S; provider manages Virtualization→Network.
- **PaaS**: user manages **Apps + Data** only; provider manages Runtime and below.
- **SaaS**: provider manages everything.
- Analogy: roads = infrastructure; cars/trucks = platform; goods/people = software.

SaaS / PaaS / IaaS – When to use

- **SaaS**: web access, central management, **one-to-many** model, no patching by user, APIs for integration. Good for: short-term need, demand spikes (tax/billing), web/mobile access. **Not good**: real-time fast processing, regulation forbids external hosting, existing on-premise suffices.
- **PaaS**: develop/test/deploy/host in same IDE; **multi-tenant**; built-in scalability/load balancing/failover; collaboration tools; billing mgmt. Good for multiple developers, agile, automated testing. **Not ideal**: need portability, proprietary languages (**vendor lock-in**), need hardware customization.
- **IaaS**: resources as a service, dynamic scaling, **utility pricing**, multiple users on one hardware. Good for volatile demand, start-ups without capital, rapid growth, **CapEx → OpEx**, temporary needs. **Not best**: regulatory compliance on offshoring, highest performance needs.

Networking & NFV

- Cloud networking concepts: **VLAN, VPN**, protocol layers; tools: **OpenSSH, OpenVPN**.
- **NFV (Network Function Virtualization)**: implement network functions in **software** on industry-standard servers/switches/storage; can be moved/instantiated anywhere without new equipment.

WEEK 2

Deployment Models, Virtualization, XML, Web Services & SOA

2.1 Deployment Models (NIST)

Public Cloud

- Infrastructure for **open use by general public**; exists **on premises of the provider**. Examples: **Google App Engine, Microsoft Azure, IBM SmartCloud, Amazon EC2**.
- Workload location **hidden**; provider may migrate workloads anywhere (cheap data centres) unless location-restriction policy offered.
- **Multi-tenancy risk**: co-resident with competitors/attackers → reliability and security risk.
- **Network dependency** on public Internet (DNS, routers, links).
- **Limited visibility/control** – proprietary operation; subscriber **cannot verify complete data deletion**.
- **Elasticity** – illusion of unlimited resources; **low up-front cost**; **restrictive default SLAs**.

Private Cloud

- Exclusive use by **single organization** (multiple consumers/business units); may be **on or off premises**, managed by org/third party. Examples: **Eucalyptus, Ubuntu Enterprise Cloud (UEC), Amazon VPC, VMware Cloud Infrastructure Suite, Microsoft ECI**.
- Two scenarios: **On-site private** (at customer premises) and **Outsourced private** (server side hosted by hosting company).
- **On-site private**: one security perimeter around subscriber + cloud; still needs **IT skills**; strong security from external threats; **significant-to-high up-front cost**; **limited (fixed) resources**; bulk import/export limited by own network.
- **Outsourced private**: **two security perimeters** joined by **protected communications link**; security depends on both perimeters + link; **modest-to-significant up-front cost** (SLA negotiation, network upgrade, porting, training); **extensive resources** can be rented.

Community Cloud

- Exclusive use by a **community with shared concerns** (mission, security, policy, compliance). Examples: **Google Apps for Government, Microsoft Government Community Cloud**.
- Participant orgs may provide, consume, or both; **at least one must provide** cloud services; each implements its own security perimeter.
- Access policy can be complex (N members) – use **RBAC / attribute-based access control**.
- On-site community: up-front costs **highly variable** (depends on provide vs consume only). Outsourced community: multiple protected links to provider; similar to outsourced private.

Hybrid Cloud

- Composition of **two or more distinct clouds** (private, community, public) that **remain unique entities** but bound by standardized/proprietary tech enabling **data and application portability**. Examples: **Windows Azure, VMware vCloud**.
- Can be extremely **complex**; may change over time as constituent clouds join/leave.

2.2 IaaS Details

- Subscriber gets **virtual computers, network-accessible storage, network components (firewalls), configuration services**.
- Fees: per **CPU hour**, GB stored/hour, bandwidth, IP addresses/hour, value-added services (monitoring, auto-scaling).
- **Scope of control**: provider controls **physical hardware + hypervisor**; subscriber controls **Guest OS, middleware, applications**.
- Hypervisor/VMM = lower privileged layer; guest OS = higher layer. VM = "an efficient, isolated duplicate of a real machine".

IaaS Cloud Architecture (3-level hierarchy)

- **Top: Cloud Manager** – public access point; sign-up, authenticating subscribers, credentials, top-level resource management.
- **Middle: Cluster Manager** – manages cluster of computers on high-speed LAN; may be geographically distributed; connected to **Persistent Local Storage (PLS)**.
- **Bottom: Computer Manager** – runs on each host; uses hypervisor interface to **start, stop, suspend, reconfigure VMs**; tracks VMs running/startable.
- **Data Object Storage (DOS)** – stores user credentials, OS images; usually **single per cloud**.
- Commands flow **top → down**.

2.3 Virtualization

- Lets one computer do job of multiple computers by sharing hardware; run multiple OSes **simultaneously & isolated** (unlike dual-boot).
- **Popek & Goldberg (1974)** – 3 goals: **Equivalence, Resource control, Efficiency**.
- Instructions: **privileged** (trap in user mode); **sensitive** (control-sensitive / behavior-sensitive).
- **VMM can be constructed if sensitive instructions ⊆ privileged instructions**.

Server virtualization approaches (3 generations)

- **1st: Full virtualization** – dynamic **binary translation/emulation**; guest OS unaware; all hardware emulated; open source emulators **QEMU, Bochs**; VMware & Microsoft.
  - Pros: isolation, **total VM portability** (Dell→HP), can run OS of another architecture.
  - Cons: performance cost; guest at **Ring 1/3** (host holds Ring 0) so VMM must trap/emulate.
- **2nd: Para-virtualization** – **guest OS modified** (recompile kernel or para-virtual drivers), uses **specialized API/hypercalls** to VMM; no binary translation; e.g., **Xen**, VMware. Needs modified OS (Novell yes, Microsoft no).
- **3rd: Hardware-assisted** – silicon-based; guest OS at **Ring 0**; VMM in more-privileged "**Ring −1**"; uses **Intel VT / AMD-V**; runs **unmodified OS** (legacy OK). Con: speed/flexibility (guest unaware).

Network Virtualization

- Making one physical network appear as multiple logical ones.
- Related concepts: **VPN** (not customizable enough), **Active & Programmable networks**, **Overlay networks** (application-layer, not flexible enough).
- Business-model players: **Infrastructure Providers (InPs)**, **Service Providers (SPs)**, **End Users**, **Brokers**.
- Design principles: concurrence of multiple heterogeneous VNs, recursion, inheritance, revisitation.
- Design goals: **flexibility, manageability, scalability, security/privacy/isolation, programmability, heterogeneity, experimental/deployment facility (PlanetLab, GENI, VINI), legacy support**.

2.4 XML (eXtensible Markup Language)

- Syntax for encoding **text-based data**; **extensible** (own tags), **self-describing**, hierarchical, universal format; designed for distributed environments; separates content from presentation. Developed from **SGML**; W3C.
- **XML vs HTML**: HTML = how to display; XML = what the data is (machine & human readable); XML tags **not predefined**.
- **Well-formed** = syntactically correct (minimum to be XML; parser must stop otherwise). **Valid** = well-formed + conforms to DTD/Schema.
- Structure: XML declaration, elements/tags, attributes, **tree data model**.
- **Entities**: internal & external (SYSTEM + URL), referenced via &name;. Character references e.g. ♠.
- **Validating parser**: must retrieve all entities/DTD; fails if cannot. **Non-validating**: stops DTD processing at first missing entity but not an error; **non-validating is always faster**.
- **DTD** (part of core XML) vs **XML Schema (2001)**: Schema uses pure XML, richer data types (integer, date, ranges); **only DTD can declare entities**.
- **Namespaces**: identify different vocabularies (`<key/>` security vs database); `xmlns` attribute with URL (URL need not point anywhere).

XML Processing APIs

- **SAX** – event-based, fast, **low memory**, stream processing; but programmer does everything.
- **DOM** – in-memory **tree**; good for dynamic modification/querying; **slower, more memory**; same interface across languages.
- **JDOM** – Java-specific, nicer OO interface than DOM. **dom4j** – Java framework; supports SAX/DOM/JDOM, mixed SAX/DOM.
- **XSLT** – XML language to **transform** XML (tree transformation) using a style sheet.
- **Parsers**: Xerces (Apache), XML4j (IBM).

XML Messaging

- XML as message format; transports: HTTP, HTTPS, SMTP, MQ, IIOP, JMS. Disadvantage: asynchronous, no delivery guarantee.
- **XML-RPC** – simple encoding of method call & parameters. **SOAP** – more complex wrapper.

2.5 Web Services & SOA

- **Web Service**: application component that communicates via open protocols, processes **XML messages framed using SOAP**, describes messages with **XML Schema**, endpoint described by **WSDL**, discovered via **UDDI**.

- Evolution: structured programming → OOP → distributed computing → **EDI** → WWW → Web services.
- **EDI**: computer-to-computer exchange of business documents in standard formats; Web services similar but **simpler & cheaper**, can work alongside EDI.
- Problems solved: **Interoperability** (DCOM bound to Windows, RMI bound to Java), **Firewall traversal** (CORBA/DCOM use non-standard ports; WS use **HTTP port 80**), **Complexity**.
- Text-based, so easier to debug.

SOA (IBM model) – 3 entities

- **Service provider**, **service requestor**, **service broker/registry**.
- Operations: **Publish**, **Find**, **Bind**.

WS Components

- **XML** (data representation), **SOAP** (communication), **WSDL** (describe services), **UDDI** (register/locate).
- Steps: client queries UDDI registry → registry refers to WSDL doc → client accesses WSDL → client sends SOAP request → service returns SOAP response.

SOAP

- **Simple Object Access Protocol**; XML-based; platform & language independent; mainly over **HTTP**; **stateless, one-way**.
- Building blocks: **Envelope (required)**, **Header (optional)**, **Body (required)**, **Fault (optional)**.
- Why SOAP: Unix RPC, CORBA, RMI, DCOM require specific platforms; SOAP is platform neutral.
- Characteristics: **Extensibility, Neutrality (HTTP/SMTP/TCP), Independence**.
- Security: HTTP over SSL; **WS-Security** defines complete encryption system.

WSDL

- XML vocabulary describing web services = **contract** between service and client; says where service is and what protocol.
- Main elements: **portType** (operations), **message**, **types**, **binding** (protocols).

UDDI

- Framework for XML-based **registries**; like **telephone yellow pages**; publish & find services; interrogated using SOAP; provides WSDL docs.
- WS Security stack: WS-Security, WS-Policy, WS-Trust, WS-Privacy, WS-Federation, WS-Authorization, WS-SecureConversation.

WEEK 3

SLA, Cloud Economics, Data Management (GFS/BigTable/Dynamo), MapReduce Intro & OpenStack

3.1 Service Level Agreement (SLA)

Basics

- SLA = **formal contract between Service Provider (SP) and Service Consumer (SC)**; foundation of consumer's trust; defines formal basis for **performance and availability** the SP guarantees.
- SLA contains **Service Level Objectives (SLOs) = objectively measurable conditions** for the service. SLA & SLO = basis for **selecting a cloud provider**.
- SLO examples: availability of service X is 99.9%; response time of query Q between 3–5 s; throughput of server S at peak load.
- SLO covers QoS parameters: availability, serviceability, billing, penalties, throughput, response time, quality.
- **Service Level Management** = monitoring/measuring performance of services based on SLOs.

SLA Contents

- Set of services; definition of each service; responsibilities of provider & consumer; **metrics**; **auditing mechanism**; **remedies** if terms not met; how SLA changes over time.

Web Service SLA languages

- **WS-Agreement** – XML-based language/protocol for negotiating, establishing, managing agreements at runtime; request-response; helps discover compatible providers.
- **WSLA (Web Service Level Agreement framework)** – XML-schema-based language + runtime interpreter; monitors QoS and reports violations; **lacks formal semantics of metrics**.

Cloud SLA vs Web-Service SLA

- WS: response time, violation rate, availability, cost. **Cloud: security, privacy, trust, management**.
- WS: negotiation/provisioning/monitoring **not automated**; **Cloud: SLA automation required** (dynamic, scalable).
- WS resource discovery via **UDDI**.

Types of SLA

- **Off-the-shelf (non-negotiable / direct) SLA** – provider creates template; used by present-day clouds; not suited for mission-critical apps.
- **Negotiable SLA** – via one or multiple external agents.

SLA Considerations & Requirements

- Business-level objectives, responsibilities of both parties, **business continuity & disaster recovery**, **system redundancy**, maintenance, **location of data**, **seizure of data** (multi-tenancy → others affected).
- Requirements: security, **data encryption**, privacy (isolation in multi-tenancy), data retention & deletion, hardware erasure, **regulatory compliance**, transparency (notify breaches), certification, **monitoring (neutral third party removes conflict of interest)**, **auditability**.

KPIs and Metrics

- **KPIs = low-level resource metrics** (downtime, uptime, inbytes, outbytes, packet size) aggregated into high-level SLOs.
- **Availability A = 1 − (downtime/uptime)** (as given in slides).
- Metrics: **Throughput**, **Availability** (% uptime in observation period), **Reliability** (how often available), **Load balancing** (when elasticity kicks in), **Durability** (likelihood data is not lost), **Elasticity**, **Linearity** (performance as load increases), **Agility**, **Automation** (% requests handled without humans), customer service response time, **service-level violation rate**, transaction time.

Example Cloud SLAs

- **Amazon EC2 (IaaS): 99.95%**; **Amazon S3: 99.9%**; **SimpleDB: no SLA**; Salesforce CRM: no SLA guarantee.
- **Microsoft Azure Compute: 99.95%**; **Azure Storage: 99.9%**.
- **Rackspace**: internal network 100%, data centre infrastructure 100%, load balancers 99.9%; server migration notified 24 h ahead & done in max 3 h; recovery within 1 h.
- **Terremark vCloud Express**: 100% monthly uptime. **Nirvanix**: 99.9%.

Limitations of SLAs

- Measured only as **uptime percentage** (ignores stability, capacity); **biased towards vendors**; **lack of active monitoring on customer side** (customers use ticketing, must report outages themselves); gap between QoS hype and reality; **no formal way to verify compliance**.

Expected SLA parameters

- **IaaS**: CPU capacity, memory, boot time, storage, scale up/down, auto-scaling, availability, response time.
- **PaaS**: integration, scalability, billing, deployment environment. **SaaS**: reliability, usability, scalability, availability, customizability, response time.
- **Storage-aaS**: location, space, billing, security, backup, fault tolerance, recovery, throughput, bandwidth, data life-cycle.

3.2 Cloud Economics

Economic properties (mnemonic CLOUD)

- **C**ommon infrastructure (pooled, standardized; **statistical multiplexing**), **L**ocation-independence (latency reduction, user experience), **O**nline connectivity, **U**tility pricing (pay-per-use; beneficial under **variable demand**), on-**D**emand resources (elastic provisioning without delay/cost).

Value of common infrastructure

- Economies of scale (lower overhead, buyer power); **statistics of scale** – multiplexing demand → higher utilization.
- **Coefficient of Variation Cv = σ / |μ|** (std dev ÷ mean) – measure of "smoothness"; not variance or correlation coefficient.
- Aggregating **n independent identical demands reduces Cv by factor √n** (Cv/√n); aggregating **100 workloads → penalty ~10%**.
- **Negatively correlated** demands (X and 1−X) smooth best; **perfectly correlated** demands → Cv unchanged; simultaneous peaks hurt.
- Midsize providers can achieve statistical economies similar to huge ones when demands independent.

Value of location independence

- Latency strongly correlated with **distance**; speed of light in fibre ≈ 124 miles/ms; VoIP with ≥200 ms latency is poor. Global user base needs dispersed architecture (consistency, availability, partition tolerance).

Value of utility pricing

- Cloud need not be cheaper per unit to be economical (rental car analogy: buy/lease ₹10,000/day vs rent ₹45,000/day → **U = 4.5**).
- Notation: D(t) demand; **P** peak; **A** average; **B** baseline (owned) unit cost; **C** cloud unit cost; **U = C/B** utility premium.
- **C_T = U·B·∫D(t)dt = A·U·B·T**; **B_T = P·B·T** (owned must handle peak).
- **Cloud is cheaper when U < P/A** (utility premium < peak-to-average demand ratio).
- Real world: demand spiky (Slashdot effect, tax season, Christmas) → **hybrid model** often best; also consider network cost, interoperability overhead, reliability.

Value of on-demand services

- **Penalty cost ∝ ∫|D(t) − R(t)| dt** (pay for unused resources or suffer missed service).
- Flat demand → penalty 0; linear demand → periodic provisioning acceptable; **exponential demand D(t)=e^t** with provisioning delay t_p → R(t)=e^(t−t_p); gap = $k_1 e^t$ (grows exponentially).

3.3 Managing Data in the Cloud

Relational DB & storage

- RDBMS default since 80s; efficient for transaction processing (System R, Ingres). SQL parser → memory/disk operations; DB file system independent of OS file system for full control of page retention.
- **Row-oriented storage**: best for write/transaction processing; B+-tree indexes. **Column-oriented storage**: best for **data warehouse** workloads (e.g., Vertica).

Parallel DB architectures

- **Shared memory** (SMP, multi-CPU servers); **Shared nothing** (independent servers, own disks, network); **Shared disk** (hybrid; shared storage via **NAS/SAN**).
- Parallel DB advantages: parallel SQL, partitioned tables, distributed joins, **two-phase commit**, fault tolerance. Examples: Oracle, DB2, SQL Server; warehouses: **Netezza, Vertica, Teradata**.

Google File System (GFS) / HDFS

- GFS manages large files on commodity servers; fault tolerance a necessity: **p(system failure) = 1 − (1 − p(component failure))^N**. HDFS = open source GFS.
- **Single Master** (GFS) = **Name Node** (HDFS) holds namespace/metadata. Files split into **chunks (GFS) / blocks (HDFS)**, typically **64 MB**, stored on **chunk servers (GFS) / data nodes (HDFS)**, **replicated 3 times** on different racks/network segments.
- **Read**: client sends path + offset to master → master returns metadata of a replica → client **caches metadata** → reads from chunk server.

- **Write/Append**: master returns all replicas → client sends data to all → master designates one **primary** → primary picks offset → informs replicas.
- **Fault tolerance**: master–chunk server **heartbeat messages**; new primary assigned on failure.

BigTable

- Distributed structured storage on GFS; **sparse, persistent, multi-dimensional sorted map**; accessed by **row key, column key, timestamp**; column = **column-family:label**; families fixed at creation, labels dynamic; cells store multiple **timestamped versions**.
- Table split into **tablets** (row ranges) managed by **tablet servers**; each column family of a tablet stored in an **SSTable**; **metadata table/server** locates tablets; **root tablet**.

Dynamo (Amazon)

- Simple **<key, value>** model; supports many small concurrent updates; suited for e-commerce; **independent of GFS/HDFS**.
- **MD5** 128-bit hash; keys mapped onto ring of **virtual nodes**; replicated on primary + (N−1) nodes.
- **Quorum**: read from R replicas, write to W replicas; **R + W > N → quorum consistent**; maintains **eventual consistency**.
- **Vector timestamps** for versioning; conflicts resolved by application logic; storage engines: **Berkeley DB (used by Amazon)**, MySQL.

Datastore (Google App Engine) / SimpleDB (Amazon)

- Simple transactional key-value stores; all entities in **one BigTable table**; sorted lexicographically; supports prefix & range queries; entity groups for transactions.
- Auto-created indexes: **single-property** and **kind** indexes; **composite indexes** configurable; highest-selectivity index chosen.

3.4 MapReduce Introduction

- **Parallel computing models**: shared-memory (any processor accesses any memory) vs distributed-memory (message passing).
- **Parallel efficiency** = T / (p × T_p); inefficiency from synchronization, communication overhead, load imbalance. **Scalable** = efficiency stays constant as data and processors grow.
- Word-frequency illustration: approach 2 (each processor handles n/p documents for all m words, then sums) is **more efficient & scalable** than approach 1 (each handles m/p words, reading many unneeded words).
- **MapReduce** (Google): **Map**: (k1,v1) → [(k2,v2)]; **Reduce**: (k2,[v2]) → (k2, f([v2])). **M mappers, R reducers, single master**. Mapper output: one file per reducer, sorted by key, on **local disk**; reducers fetch via **RPC**; final output to **GFS**.
- **Fault tolerance**: **heartbeat**; failed **mapper → re-executed** (output is on local disk); failed **reducer → only remaining tasks reassigned** (completed already in GFS); stragglers → duplicate tasks.
- **Efficiency**: $\varepsilon_{MR} = 1 / (1 + 2c\sigma/w)$.
- Applications: **inverted indexing**, relational operations (joins/group by) at large scale with fault tolerance. **Hadoop** = open-source MapReduce (Yahoo!), on Amazon EC2 AMIs.

3.5 OpenStack

- **Cloud operating system** controlling pools of compute, storage, networking via dashboard; started by **NASA + Rackspace**; supports IaaS (and PaaS e.g. Cloud Foundry on top, SaaS).
- Features: VMs on demand, provisioning, snapshotting, multi-tenancy, **quotas** per project/user.

Components (service → project)

- **Compute → Nova** (spawn, schedule, decommission VMs).
- **Networking → Neutron** (Network-Connectivity-as-a-Service; pluggable).
- **Object storage → Swift** (REST/HTTP; replicated; not a mountable file server).
- **Block storage → Cinder** (persistent block storage to instances).
- **Identity → Keystone** (authN/authZ; catalog of endpoints).
- **Image service → Glance** (stores/retrieves VM disk images).
- **Telemetry → Ceilometer** (metering for billing, benchmarking).
- **Dashboard → Horizon** (web self-service portal).

Workflow

- User logs into **Horizon** → Keystone issues token → Horizon sends request to **Nova API** (validates token with Keystone) → Nova API **rpc.cast to Scheduler** via message queue → Scheduler filters/selects compute node → **Nova Compute** (asks **Conductor** for VM info) → **Neutron** configures network → **Cinder** gives block device → **Glance** gives image URI → image retrieved via **Swift** → hypervisor runs VM.

Storage concepts

- **Ephemeral** (Nova): persists until VM terminated; local file system. **Block** (Cinder): persists until deleted; block device (/dev/vdc). **Object** (Swift): persists until deleted; accessible from anywhere; stores files incl. VM images.

WEEK 4

Private Cloud on OpenStack (Meghamala), Azure Web App, Google Cloud Platform

4.1 Private Cloud using OpenStack – "Meghamala" @ IIT Kharagpur

- **Meghamala** = IIT Kharagpur's private cloud built using **OpenStack**.
- Lifecycle shown: **Overview → VM creation → Accessing VM by user → VM termination**.
- Login through **Horizon** (web dashboard).
- Dashboard views: overview of **compute nodes**, graphical **resource usage**, details of **instances** (each VM gets internal IP e.g. 10.4.0.x and a floating/public IP), **volumes (Cinder)**, **images (Glance)**, **security group rules (Neutron)**, **hypervisor vCPU/RAM/storage (Nova)**, **flavors (Nova)**, compute services.
- Component mapping: **Nova** = compute/flavors/hypervisors; **Glance** = images; **Cinder** = volumes; **Neutron** = networks/security groups; **Keystone** = identity; **Horizon** = dashboard.
- **Flavor** = predefined VM size (vCPU, RAM, disk).
- **Security group** = set of network access rules for instances.
- Created VM was accessed using the **X2Go client** (remote desktop) – example VM named *cloud-nptel*.
- After use, the VM is **terminated** from the dashboard to release resources.

4.2 Microsoft Azure – Overview

- Growing collection of integrated cloud services to **build, deploy and manage** applications via a **global network of datacenters**.
- Highlights: deploy anywhere with your choice of tools (**consistent hybrid cloud**, open source), trusted cloud (security, privacy, compliance, transparency), accelerate app innovation (consistent portal), insights from analytics.
- **Azure Web Apps** = highly scalable, **self-patching** web hosting service (**PaaS**).
- Prerequisites for demo: **Git** and **Python**.

Steps: Create a Python (Flask) web app on Azure

- Log in to **portal.azure.com**; launch **Azure Cloud Shell** (free **bash** shell inside portal).
- **Clone** sample repo → change directory → **install Flask** → run locally (`http://localhost:5000` shows *Hello World*).
- **Configure a deployment user** (needed for **FTP and local Git** deployment): `az webapp deployment user set --user-name <u> --password <p>`.
- **Create a resource group** = logical container for Azure resources (web apps, databases, storage accounts).
- **Create an App Service plan** – defines **region, instance size (small/medium/large), scale count (1–20 instances), SKU (Free, Shared, Basic, Standard, Premium)**; multiple apps can share one plan to save money.
- **Create the web app** – hosting space + URL (`<name>.azurewebsites.net`).
- **Configure Python version** (uses a default platform container).
- **Configure local Git deployment** (App Service supports **FTP, local Git, GitHub, Visual Studio Team Services, Bitbucket**).
- **Add Azure remote** to local repo → **git push azure master** (use the deployment-user password, **not** portal password).
- To update: edit `main.py` → **commit** → **push** → refresh page.

4.3 Google Cloud Platform (GCP)

- Set of **modular cloud-based services** to build, test and deploy apps on Google's infrastructure (simple websites to complex apps).
- Why GCP: runs on Google's infrastructure (billions of searches in ms, YouTube, Gmail); **focus on product** (managed services, SDKs, console); **mix and match** (VMs, managed platform, blob/block storage, NoSQL datastore, MySQL, big-data analytics); **scale to millions** – auto **scale up and scale down**, **pay only for what you use**; consistent CPU/memory/disk performance, global network & edge cache; support.
- Services: **App Engine = PaaS** (focus on code, no patching); **Compute Engine = raw VMs (IaaS)**; **Cloud SQL = MySQL**; **Cloud Datastore = NoSQL**; **Cloud Storage = object storage with global edge caching**; Google **APIs**; **Cloud Endpoints**.

Example 1: Host a static web page on Cloud Storage

- Create a **bucket** → edit configuration / **Configure website** (set **Main page** and **404 page**) → upload files → make them **shared publicly** → use the public link (storage.googleapis.com/...).

Example 2: Python web app on App Engine

- Create **project** and App Engine application → select **region** → select **language** → activate **Cloud Shell** → clone **Hello World** sample.
- Each app needs **app.yaml** (configuration) and **main.py** (code; Flask).
- Run local dev server: **`dev_appserver.py $PWD`** (watches files & reloads).
- Deploy: **`gcloud app deploy app.yaml --project <id>`**; view: **`gcloud app browse`**.

WEEK 5

SLA / Economics / MapReduce Tutorials, Resource Management I & II

5.1 SLA Tutorial (worked problems)

Key facts

- SLA = formal contract between **Service Provider (SP) and Service Consumer (SC)**; foundation of consumer trust; contains **SLOs** (objectively measurable conditions); basis of provider selection.
- **Availability % = (Total service hours − Outage hours) / Total service hours × 100**.

Problem 1 – 99% guarantee, app runs 12 h/day for a month

- Total service time = 12 × 30 = **360 h**; allowed downtime = 1% = **3.6 h**.
- Actual outage = **10.75 h > 3.6 h** → availability ≈ **97.0%** → **provider HAS violated** the guarantee.

Problem 2 – 99.95% guarantee, 30 days, 12 h/day, $50/day

- Service credit table: uptime **< 99.95% → 10% credit**; **< 99% → 25% credit**.
- Outages: 5 h + 30 m + 1 h 30 m + 15 m + 2 h 25 m = **9 h 40 m ≈ 9.67 h**.
- Uptime = (360 − 9.67)/360 ≈ **97.3% (< 99%) → 25% credit**.
- Cost = 30 × $50 = $1500; credit = $375 → **effective cost payable = $1125**.

5.2 Cloud Economics Tutorial

Properties (CLOUD)

- **C**ommon infrastructure (pooled, standardized; **statistical multiplexing**), **L**ocation-independence, **O**nline connectivity, **U**tility pricing (**pay-per-use**, good for variable demand), on-**D**emand resources (scalable/elastic).

Utility pricing formulae

- D(t) demand; **P** = peak demand; **A** = average demand; **B** = baseline (owned) unit cost; **C** = cloud unit cost; **U = C/B** utility premium.
- **C_T = A·U·B·T**; **B_T = P·B·T** (owned capacity sized for peak).
- **Cloud cheaper iff C_T < B_T ⇔ U < P/A**.
- Real world: spiky demand (news, promotions, product launches, flash floods, tax season, Christmas) → **hybrid model** often best; also consider network cost, interoperability overhead, reliability/accessibility.

Value of on-demand services

- **Penalty cost ∝ ∫|D(t) − R(t)| dt** (pay for unused resource **or** suffer missed service).
- Flat demand → penalty 0; linear demand → periodic provisioning acceptable; **exponential demand D=e^t** with provisioning interval t_p → R=e^(t−t_p); shortfall = $k_1 e^t$ grows exponentially.

Assignment 1 (solved)

- D(t)=50 sin t (0≤t<π/2), 20 sin t (π/2≤t<π); R(t)=D(t)+δ·D′(t); δ=π/12; cost 0.9/unit/time.
- |D−R| = δ|D′|; ∫|D′|dt = 50 + 20 = 70 → **penalty = 0.9 × (π/12) × 70 ≈ 16.5 units**.

Assignment 2 (solved)

- D(t)=50(1+e^−t), P=100; T=100 → **A ≈ 50.5**; **P/A ≈ 1.98**; **U = 200/120 ≈ 1.67**.
- U < P/A → **cloud is cheaper**. (C_T ≈ 1,010,000 vs B_T = 1,200,000.)

Assignment 3 (solved) – in-house vs cloud server

- 3-year hours = 3×365×24 = 26,280. In-house cost/hr = 6,00,000/26,280 ≈ **₹22.83**.
- **Price of a core-hour**: in-house = 22.83/12 ≈ **₹1.90**; cloud = 42/8 = **₹5.25**.
- **Cost per effective hour** = cost/efficiency: in-house = 22.83/0.4 ≈ **57.08**; cloud = 42/0.8 = **52.5**.
- **Total cost/effective-hour** = cost/effective-hr + power & cooling + management: in-house = 57.08+22+6 ≈ **85.08**; cloud = 52.5+0+1 = **53.5**.
- **Ratio in-house : cloud ≈ 1.59**.
- If in-house efficiency = 70%: 22.83/0.7 + 28 ≈ **60.6** > 53.5 → **cloud still better**.

5.3 MapReduce Tutorial

- **MapReduce**: programming model from **Google**; for large-scale search/text processing on web data stored in **BigTable + GFS**; massively parallel (tens of thousands of processors); **fault tolerant**. **Hadoop** = open-source implementation (**developed at Yahoo!**), available as pre-packaged **AMIs on Amazon EC2**.
- Two phases: **Map** and **Reduce**; **M mappers, R reducers**, coordinated by a **single master**.
- **Map phase**: each mapper reads ≈ **1/M** of input from global file system (locations from master); **(k1,v1) → [(k2,v2)]**; writes **one file per reducer**, **sorted by key**, on **local file system**; master tracks file locations.

- **Reduce phase**: master tells reducers where data is; reducers fetch via **remote procedure calls**; group by same key and apply f: **(k2,[v2]) → (k2, f([v2]))**; final results written to **GFS**.
- Example: word count – 3 mappers, 2 reducers; map: (d_k,[w1…wn]) → [(w_i,c_i)]; reduce: (w_i,[c_i]) → (w_i, Σc_i).

Problems (solved)

- **P1 (HDFS blocks, 64 MB)**: files 64 KB → 1 block; 65 MB → 2 blocks; 127 MB → 2 blocks → **total 5 blocks** (a small file still uses its own block but only occupies its actual size).
- **P2 (average of integers)**: Map emits (key="avg", value=x) for each integer (better: (key,(x,1)) so a combiner can be used); Reduce sums values and counts → for A=(10,20,30,40,50): map output (avg,10),(avg,20)…; reduce: sum=150, count=5 → **average = 30**.
- **P3 (salary by gender)**: Map parses "Name, Gender, Salary" → emits (Gender, Salary); Reduce: total = Σ salaries, average = total/count per gender.
- **P4 (word-length categorization)**: Map: for each word emit (category, 1) where **tiny 1–2, small 3–5, medium 6–9, big ≥10 letters**; Reduce: sum counts per category.

5.4 Resource Management – I (Green Cloud)

- **Resource types**: **Physical** (computer, disk, database, network, scientific instruments) and **Logical** (execution, monitoring, communication application).
- **Resource management** = operations to control how capabilities of cloud resources/services are made available to users/apps/services **efficiently**.

Data-centre power consumption

- Servers ≈ **0.5% of world's electricity**; server energy demand **doubles every 5–6 years**; large CO_2 from fossil fuels; need to cut energy with **minimal performance impact**.
- **Economic motivation**: megawatt scale, millions of dollars to operate, facilities at peak and can't expand without new power source. **Environmental**: majority fossil fuels, huge CO_2, sustainable sources not ready.

Green computing

- Advanced scheduling to cut energy: **power-aware** and **thermal-aware**.
- **Performance/Watt is not following Moore's law.**
- Data-centre design to reduce **Power Usage Effectiveness (PUE)**: cooling systems, rack design.

Green Cloud Framework

- **Virtual Machine controls**: **Scheduling** (power-aware, thermal-aware) and **Management** (VM image design, migration, dynamic shutdown).
- **Data-centre design**: server & rack design, air conditioning & recirculation.

VM scheduling on multi-core systems

- **Non-linear** relationship between # active cores and power (Intel Core i7 920, 4 cores/8 virtual with Hyperthreading: ~105 W idle → ~170 W with 8 cores).
- **Power-aware scheduling**: **greedy** algorithm; schedule **as many VMs as possible on one multi-core node**, track cores per node, match VM needs to node capacity.
- Example: all 8 VMs on one node (170 W) + 3 idle nodes (105 W each) = **485 W** vs spreading 2 VMs on each of 4 nodes (138 W each) = **552 W**.

VM management

- Monitor load. **Load decreases** → **live-migrate** VMs to more-utilized nodes, **shut down unused nodes**. **Load increases** → use **Wake-on-LAN (WOL)** to start nodes, schedule new VMs.

VM image minimization

- VMs are loaded with unwanted packages/services; clouds are **service-oriented (SOA)** → need **lightweight Linux VM**; small images reduce **network latency**.
- Typical image: start from **Ubuntu 9.04**; remove packages not needed (**no X11, no window manager**, minimal server); **readahead profiling** (reorder boot sequence, prefetch files, cut CPU idle from I/O); optimized kernel (built for **Xen DomU**, no 3D/sound, modules built in).
- Energy savings: boot time **38 s → 8 s**; 30 s @ 250 W = 2.08 Wh ≈ 0.002 kWh; small cloud (100 images/h) saves ~$262.65/yr at 15.2¢/kWh; production cloud (1000 images/min) saves 120 kWh/h → **> $1 million/yr**; image size **4 GB → 635 MB** (faster live migration).
- Summary: combine power-aware + thermal-aware scheduling; integrated server/rack/cooling; small savings → large impact; combining methods has larger impact than individually.

5.5 Resource Management – II (IaaS)

- **IaaS** is the most popular cloud service; provider offers VMs, raw (block) storage, firewalls, load balancers, network devices; **resource management is a major challenge**.
- **Objectives**: scalability, quality of service, optimal utility, reduced overheads, improved throughput, reduced latency, specialized environment, cost effectiveness, simplified interface.
- **Challenges – hardware**: CPU, memory, storage, workstations, network elements, sensors/actuators. **Logical**: OS, energy, network throughput/bandwidth, load balancing, information security, delays, APIs, protocols.
- **Aspects**: provisioning, allocation, requirement mapping, adaptation, discovery, brokering, estimation, modeling (and scheduling).

Definitions

- **Provisioning** – allocation of a provider's resources to a customer.
- **Allocation** – distribution of resources economically among competing groups/programs.
- **Adaptation** – ability of system to adjust resources dynamically to meet user needs.
- **Mapping** – correspondence between resources required by users and resources available at provider.
- **Modeling** – based on detailed info on network elements/entities; states, transitions, inputs/outputs; helps **predict future requirements**.
- **Estimation** – close guess of actual resources an application needs.
- **Discovery & selection** – identify list of authenticated resources available for job submission and choose the best.
- **Brokering** – negotiation of resources through an agent so resources are available at the right time.
- **Scheduling** – timetable of events/resources (depends on duration, predecessor activities, relationships, resources allocated).

Approaches (name → key idea)

- **Provisioning**: **Nash equilibrium (game theory)**; **network queuing model** (tiers as queues); **prototype provisioning** (**k-means** clustering + queuing model); **VM provisioning** on **Xen** with **SEDF (Simple Earliest Deadline First)** scheduler & weighted fair CPU sharing; adaptive provisioning (bottleneck detection); **SLA-oriented** methods; dynamic & automated framework; **OCRP – Optimal Cloud Resource Provisioning** (demand/price uncertainty; deterministic equivalent formulation, sample-average approximation).
- **Allocation**: **market-oriented** (uses **Model Predictive Control, MPC**); **intelligent multi-agent model**; **energy-aware (ant-colony / pheromone)**; measurement-based performance analysis (co-location); dynamic allocation based on VM load; real-time allocation for small/medium IaaS; **dynamic scheduling & consolidation using DAGs**.
- **Mapping**: symmetric mapping pattern (3 functions); **load-aware mapping** (multicast image transfer, caching); minimum congestion mapping (graph mapping); **iterated local search** request partitioning; SOA API; **impatient task mapping (genetic algorithm, throughput as fitness)**; **DEVAs** (distributed ensembles of virtual appliances); mapping virtual network onto substrate network (**backbone mapping**).
- **Adaptation**: **reinforcement-learning guided control policy** (MIMO feedback control); web-service-based prototype; **OnTimeMeasure** service (thin-client virtual desktop clouds); virtual networks architecture; **DNS-based load balancing**; **hybrid approach** (layered queuing models for Xen, **bin packing + gradient search**).
- **Performance metrics for resource management**: **reliability, ease of deployment, QoS, delay, control overhead**.

WEEK 6

Cloud Security I–IV & Broker for Cloud Marketplace

6.1 Cloud Security I – Basics of Security

Basic components (CIA)

- **Confidentiality** – keeping data/resources hidden. **Integrity** – data integrity + **origin integrity (authentication)**. **Availability** – enabling access to data/resources.

Security attacks (4 types) – source S → destination D

- **Interruption** → attack on **availability**.
- **Interception** → attack on **confidentiality**.
- **Modification** → attack on **integrity**.
- **Fabrication** → attack on **authenticity**.

Passive vs active attacks

- **Passive**: eavesdropping; **release of message contents** and **traffic analysis**; very hard to detect (prevent rather than detect).
- **Active**: **masquerade** (pretend to be another entity), **replay** (capture and retransmit), **modification** of message, **denial of service**.

Classes of threats

- **Disclosure** (snooping), **Deception** (modification, spoofing, repudiation of origin, denial of receipt), **Disruption** (modification), **Usurpation** (modification, spoofing, delay, DoS).

Policies, mechanisms, goals

- **Policy** says what is/is not allowed; **mechanisms** enforce policy. Conflicting policies → vulnerabilities.
- Security goals: **Prevention, Detection, Recovery** (continue to function even if attack succeeds).
- **Assurance**: specification → design → implementation.
- Operational issues: cost-benefit analysis, risk analysis, laws & customs. Human issues: organizational problems, **insiders/outsiders, social engineering**.

Security services

- Confidentiality, **authentication**, integrity, **non-repudiation** (order is final), **access control**, availability.
- Role of security infrastructure: confidentiality, integrity, availability, authentication (identify users), **authorization** (is operation allowed?), non-repudiation, safety.

Types of attack

- Social engineering/phishing, physical break-ins, password attacks, **buffer overflows**, **command injection**, DoS, faulty application logic, snooping, packet manipulation/fabrication, backdoors.

Network security – 6 steps

- **1. Determine security policy** (usage, training, privacy policy, update/audit schedule; DMZ & firewall placement).
- **2. Implement policy**: install **firewalls** (**iptables** on Linux; default **drop** incoming) and **IDS** (**snort**).
  - Firewall types: **Packet filter** (dest IP/port/protocol), **Stateful** (tracks TCP sessions; drops out-of-session packets), **Application proxy** (scans all layers for malicious data).
  - **IDS** scans incoming messages and creates alerts.
- **3. Reconnaissance**: find hosts, key servers, services, vulnerabilities; **passive = undetectable**, **active = often detectable by IDS**.
- **4. Vulnerability scanning**: **Nessus** (detects, produces risk report); **Metasploit** (exploit; choose **payload** = code that runs once inside).
- **5. Penetration testing**: exploit vulnerabilities using frameworks (Metasploit).
- **6. Post-attack investigation**: forensics; maintain **chain of evidence**; examine **low-level copy of disk**, not original; usually by third party.

6.2 Cloud Security II – Threats & Risks

- Cloud = outsourcing; **infinite elastic scalability, on-demand just-in-time provisioning, no upfront cost (reduces CAPEX), pay-as-you-go**.
- Providers benefit from higher datacenter **utilization**.
- Clouds still face traditional **confidentiality, integrity, availability, privacy** issues + additional attacks. (IDC survey: **security** top barrier.)

Why cloud brings new threats

- **Co-tenancy**: attacker can legitimately sit on the same physical machine as target; customer **lacks control** over data & apps; **reputation fate-sharing**.

Security stack / responsibilities

- **IaaS**: provider secures facilities→hardware (customer secures more). **PaaS**: provider handles platform & infrastructure security. **SaaS**: provider responsible for most (service levels, security, governance, compliance, liability).
- Moving IaaS → SaaS: **provider's security responsibility increases, customer's decreases**.

Gartner's Seven Cloud Security Risks

- **1 Privileged user access** (ask about hiring/oversight of admins). **2 Regulatory compliance & audit** (providers refusing audits signal "trivial use only"). **3 Data location** (jurisdiction, cross-border laws, SOX, HIPAA, subcontracting). **4 Data segregation** (encryption not a cure-all; key store protection). **5 Recovery** (replicate across sites; **RPO** = max data loss, **RTO** = max time to restore). **6 Investigative support** (logs co-located, spread across hosts). **7 Long-term viability** (data retrieval format; switching causes: price rise, bankruptcy, shutdown, quality drop, dispute; **vendor lock-in**).

Other security issues

- **Virtualization**: components **VM and VMM/hypervisor**; major function **resource isolation**.
  - Hypervisor vulnerabilities: **shared clipboard**, **keystroke logging**, **VM backdoors (covert channel)**, **ARP poisoning**.
  - Hypervisor risks: **rogue hypervisor rootkits**, external modification of hypervisor, **VM escape** (malicious code bypasses VM to get root on host), DoS.
- **Access control & identity management (IDM)**: authenticate users/services; identity theft; privacy via data mining.
- **Application security** (web-service based): **injection attacks**, **XML Signature Element Wrapping** (body moved into wrapper in SOAP header), **Cross-Site Scripting (XSS)**, flooding attacks.
- **Data life-cycle management**: confidentiality, integrity (is provider computing/storing correctly?), availability (DoS, provider going out of business), **data location**, archive, access latency.

6.3 Cloud Security III – Cross-VM Attacks (Ristenpart et al., CCS 2009)

- Paper: **"Hey, You, Get Off of My Cloud!"** – first work on **cloud cartography**; attack on **Amazon EC2**; up to **40% success** in co-residence with target VM.
- New risks: trust & dependence on provider; **multi-tenancy** (VMs of different customers on same hardware).
- Side-channels: cross-VM leakage through shared **CPU data caches**; can extract **RSA/AES keys**.

Attack model – two steps

- **Placement** (get malicious VM on same physical machine as victim) and **Extraction** (side-channel attack).
- Threat model: provider trusted; no insider/admin or hypervisor-bug attacks; attacker is a normal paying customer (max **20 instances** in parallel per EC2 account).

Four questions

- Q1 can one **locate** an instance in cloud? Q2 can one detect **co-residence**? Q3 can adversary **cause co-residence**? Q4 can adversary **exploit** co-residence?

Amazon EC2 facts

- **Xen hypervisor**; **Dom0** = privileged VM managing guests (appears as a hop in traceroute). Three degrees of freedom: **instance type, region, availability zone**. Instance has internal & external IP.

Q1 – Cloud cartography

- Hypothesis: different **availability zones and instance types map to different IP ranges**; EC2 internal address space cleanly partitioned by zone. Tools: **nmap** (TCP connect probe), **hping** (TCP SYN traceroute), **wget**, **WHOIS** (find EC2 prefixes).

Q2 – Determining co-residence (network-based checks)

- Likely co-resident if: **matching Dom0 IP** (last hop in SYN traceroute), **small packet round-trip times**, **numerically close internal IPs**. Verified using a **hard-disk-based covert channel**. Efficient check needs at most **two TCP SYN packets** ("quiet").

Q3 – Causing co-residence

- **Brute-force placement**: 141 of 1686 victims (**8.4%** coverage).
- **Placement locality**: **sequential** (one instance launched after another terminates, often same machine) and **parallel**. **Instance flooding**: launch many instances right after victim launches → **40% success with just 20 probes**.

Q4 – Exploiting co-residence

- Covert channels, **cache-usage measurement (Prime–Trigger–Probe)**, **load-based co-residence check**, **estimating web-traffic rate** of co-resident site, **keystroke timing attack** (inter-keystroke timing → password recovery).

Preventive measures

- **Randomize IP allocation**, block nmap/traceroute, hide Dom0, **disallow co-residence** (good for user but inefficient for provider); **side-channel leakage: no complete solution**.

6.4 Cloud Security IV – Secure Collaboration in SaaS Clouds

- Unique features: **co-tenancy** and **lack of control** over outsourced data/apps. SaaS = **maximum lack of control** (no active data streams/audit trails/outage reports).
- Collaboration types: **tightly-coupled (federated)** vs **loosely-coupled** (dynamic sharing; security mechanisms mostly designed for tightly-coupled).
- Goal: select an **ideal SaaS provider** and secure **loosely-coupled collaboration** in it. Four objectives:
  - **I. SelCSP** – framework to select a **trustworthy and competent** collaboration provider (trust/reputation/competence; standard SLA parameters).
  - **II. Recommend access requests from anonymous users** using **Risk-based Access Control (RAC)** with **fuzzy inference system** – balances **access risk vs security uncertainty**; more flexible than binary **MLS**.
  - **III. Inter-Domain Role Mapping (IDRM)** – find a **minimal set of roles** covering requested permissions; **no polynomial-time solution** → greedy heuristics; variants **IDRM-safety** and **IDRM-availability**; goal: minimal **excess privilege**.
  - **IV. Dynamic detection & removal of access-policy conflicts** (distributed, uses local info): conflicts = **cyclic inheritance** and **violation of Separation of Duty (SoD)**; uses **role sequences/role cycles (safe & unsafe)**; removal via **RBAC hybrid hierarchy (I-, A-, IA-hierarchy)** or introducing a **virtual role**.

6.5 Broker for Cloud Marketplace

- Need: many providers with varying QoS → intelligent **"middle man" broker** to suggest best provider and safeguard customer.
- Motivation: flexible provider selection, trustworthiness, monitoring, **avoiding vendor lock-in**.
- Objectives: select most suitable provider for QoS requirements; compute **degree of SLA satisfaction & trustworthiness**; **dynamic service migration** decisions.
- Related approaches: **CloudCmp** (compares providers' QoS), fuzzy provider selection, satisfaction-measure frameworks, trust/competence-based selection.
- Architecture: **Provider selection** by **fuzzy inference engine** (inputs: offered QoS + trust; output: suitability; request goes to max suitability) → **Monitoring module** (third party/monitoring tool) → **Migration decider** (fuzzy; output **degree of SLA satisfaction**; **migrate if below threshold**) → **Migration module** (selects target provider).
- Case studies (IaaS and SaaS): **10 providers, 500 requests, year-long simulation**, degraded QoS follows **Gaussian distribution**; compared with **conventional minimum-cost crisp broker** (fuzzy broker performs better).

WEEK 7

Mobile Cloud Computing, Fog Computing, Geospatial Cloud

7.1 Mobile Cloud Computing (MCC)

Basics

- Motivation: growth of smartphones/apps; mobile devices have **resource limits (battery, storage, bandwidth)**; cloud offers infrastructure/platform/software on demand.
- **MCC = combination of cloud computing + mobile computing + wireless networks** to bring rich computational resources to mobile users; removes need for powerful device (CPU, memory).
- **Mobile Backend-as-a-Service (MBaaS)**: lets mobile app developers connect apps to backend cloud storage/processing; abstracts infrastructure; useful for multiple apps/platforms/developers.
- Examples of cloud augmenting mobiles: **Amazon Silk** (split browser), **Apple Siri** (speech recognition in cloud), **Apple iCloud** (storage & sync), image recognition / augmented reality (Google Glass).
- Why MCC: speed & flexibility, shared resources (data-intensive work in cloud), integrated data, fewer device resources used.
- **Pros**: saves battery, faster execution. **Cons**: must send program state (consumes battery), **network latency** can delay execution.
- MCC = framework to **augment a resource-constrained device by executing parts of the program on cloud servers**.

Architecture

- Mobile devices connect via **base stations** to mobile networks → requests go to **central processors/servers** providing mobile network services → delivered to the **cloud through Internet → cloud controllers** process requests and provide services.

Key components of an offloading system

- **Profiler** – monitors execution (time, power consumption, network traffic).
- **Solver** – decides which parts run on mobile vs cloud (partitioning).
- **Synchronizer** – collects/combines results of split execution, keeps it transparent to the user.
- **Dynamic runtime offloading**: application profiling, runtime partitioning, migration of intensive components, continuous synchronization.

Advantages of MCC

- **Extends battery life** (computation offloading), improves **data storage capacity & processing power**, improves **reliability & availability** (data in cloud; virus scanning, authentication), **dynamic provisioning, scalability, multi-tenancy, ease of integration**.

Challenges

- **Security for mobile users** (malicious code, GPS privacy; move threat detection to cloud) and **securing data on clouds**.
- **Context-aware mobile cloud services** (use data types, network status, device environment, user preferences to improve QoS).
- Network access management; **QoS (network delay)** – **CloneCloud and Cloudlets** reduce delay; **standard interface/interoperability**; **service convergence**.
- Communication issues: **low bandwidth** (biggest issue), service availability, heterogeneity of networks.

MCC systems

- **MAUI (Mobile Assistance Using Infrastructure)**: programmer marks methods as **"remoteable"**; **native methods cannot be remoteable**; MAUI server in cloud; profiler + solver decide at runtime to save **energy and time**.
- **COMET (Code Offload by Migrating Execution Transparently)**: works on **unmodified applications** (only binaries); **threads migrate**; runs multi-threaded programs correctly; sends only **modified parts of the heap**.

Task partitioning problem

- Core problem: partition a program across heterogeneous resources = classic **task partitioning / job scheduling problem**.
- Input: **call graph (DAG)** with per-node energy to run locally and energy to transfer state. Output: two sets (run on mobile / run on cloud).
- **Native tasks** must run on the mobile (e.g., v1, v4, v9). Formulated as **0-1 integer linear program (ILP)**: I_v = 0 local, 1 cloud. NP-hard → heuristics (no performance guarantee).
- **Static partitioning**: ILP solved at app launch. **Dynamic/adaptive partitioning**: conditions and input change for long-running programs.

Cloudlet

- **Cloudlet** = trusted, resource-rich computer/cluster, well connected to the Internet, available to **nearby mobile devices**; **middle tier of 3-tier hierarchy: mobile device – cloudlet – cloud**.
- Reduces latency (single-hop, Wi-Fi) and lowers battery use.

When to offload? (energy model)

- Energy saved = $\mathbf{P_c \times C/M - P_i \times C/S - P_{tr} \times D/B}$, where C = instructions to compute, M = mobile speed, S = server speed ($\mathbf{S = F \times M}$, F = how many times faster), D = data transmitted, B = bandwidth, $\mathbf{P_c}$ = power while computing on phone, $\mathbf{P_i}$ = power while idle (waiting), $\mathbf{P_{tr}}$ = power while transmitting.
- Rewritten: $\mathbf{(C/M)\cdot(P_c - P_i/F) - P_{tr}\cdot(D/B)}$.
- **Offloading beneficial when large computation C with small communication D** (D/B small compared with C/M and F large). Not all apps save energy in the cloud.
- Offloading approaches: partition by estimated energy before execution; dynamic partitioning using runtime trade-off; profile-based cost graph.

Evaluating MCC

- **Energy consumption, time to completion, monetary cost, security.**
- Open questions: practical framework; scalable partitioning (NP-hard); relevant parameters.

Applications

- **Mobile healthcare**, **mobile gaming** (offload rendering), **mobile commerce**, **mobile learning** (cloud-based m-learning), assistive apps for blind (pedestrian crossing guide, currency reader).

MuSIC (Mobility-aware optimal Service allocation In mobile Cloud computing)

- Handles **user mobility**; multiple QoS goals (delay, power, cost). Uses **tiered cloud architecture**: **Tier 1 = public clouds** (EC2, Azure, App Engine – scalable but lacking fine location granularity); local/cloudlet tiers closer to user.
- Modelling: **Location-Time Workflow (LTW)** = application workflow + user trajectory; model includes cloud service set, local cloud capacity, location map; **MAPCloud analytics** from user log DB and QoS-aware service DB.

Case study – Context-aware dynamic parking service

- MCC + vehicular networks (context-aware vehicular CPS). Parking-space context from **WSN** → traffic cloud → users. **Dynamic parking**: temporary roadside parking permitted when traffic capacity allows (decided by traffic authorities using historical flow, road & weather conditions); reservation service; **context-aware optimization** uses expected parking duration.

7.2 Fog Computing

Why fog?

- Cloud-only problems: **high latency** (e.g., accident-location alerts via distant datacenter), **core-network congestion** from centralized datacenters, high bandwidth need; unsuitable for **real-time** apps.
- Cloud features: dynamic scalability, no infrastructure management by user, metered service.

Definition

- **Fog computing (fogging/edge computing)** – data, processing and applications are concentrated in **devices at the network edge** rather than almost entirely in the cloud. Term **introduced by Cisco** (for IoT).
- Cellular base stations, routers, Wi-Fi gateways can run applications; end devices (sensors) do basic processing; **lower response time → real-time apps**; reduces bandwidth by **aggregating data at access points**.
- Fog **extends cloud and services to the edge**; **not a replacement** for cloud – fog + cloud handshake needed.
- **Fog enablers**: **virtualization** (VMs on edge), **containers (e.g., Docker – lightweight)**, **Service-Oriented Architecture**.
- **Benefits**: **low latency & location awareness**, widespread geographic distribution, **mobility support**, very large number of nodes, predominant role of wireless access, real-time interactions, heterogeneity.
- Fog vs cloud: fog distinguished by **proximity to end users**, dense geographic distribution, mobility.
- Limitations of cloud solved by fog: reduce data movement/congestion, eliminate centralized bottlenecks, improved security of encrypted data closer to user.

Use-cases & applicability

- Emergency evacuation, natural-disaster management (landslide/flash-flood alerts), large sensor deployments (pre-process/summarize before sending to cloud).
- **Smart grids, smart traffic lights, connected vehicles (CV), wireless sensor networks, IoT, Software Defined Networks.**
- Connected vehicle: cars-to-cars, cars-to-access-points (Wi-Fi, 3G, LTE, **RSUs – roadside units**, smart traffic lights).

Fog challenges & resource management

- Resource allocation among applications while ensuring **end-to-end latency**; security.
- Resource-management goals: utilize idle fog nodes, more parallelism, **load balancing**, meet real-time delay requirements, **crash fault tolerance**, scalability.
- Challenges: data may not be at executing node (**data fetching**); node may become unresponsive; **micro-service execution migration** to new node; **state migration** of partially processed data; deploy components to meet latency.
- Approaches: execution migration to nearest node; minimize **carbon footprint** for video streaming; resource prediction/estimation/reservation & pricing; **Docker as edge platform**; relinquish-probability-based management; base-station association/VM placement (MINLP) for fog medical CPS.

Fog security

- **Authentication** at gateways and fog nodes, **Man-in-the-Middle attack**, privacy. Smart-grid example: smart meters have IPs; malicious user can **tamper with meter, report false readings, spoof IPs**.

7.3 Use Case: Geospatial Cloud (IIT Kharagpur Geo-Cloud)

- **Geographic information** = info explicitly linked to locations on Earth; **static** (e.g., lake, town) vs **dynamic** (e.g., population). Types: legal, political, cultural, climatic, topographic, biotic, medical, economic, infrastructure.
- Data sources: social surveys, natural surveys (**SOI maps**), **remote sensing** (air photos, satellite), reporting networks (weather stations), **field data (GPS)**.
- **GIS** = computer system for **capturing, storing, querying, analyzing, displaying** geospatial data. Components: **hardware, software, data management/analysis procedures, spatial data, people**.
- GIS challenges: **data intensive, computation intensive, variable load** (needs dynamic scaling), high reliability, network-intensive web services.
- **Heterogeneity** of GIS layers across departments → need homogeneous data description, standard encoding, standard sharing mechanism.
- **SDI (Spatial Data Infrastructure)** – coordination basis for discovery, evaluation, application of spatial data. Interoperable, **service-driven GIS**.
- **Need for geospatial cloud**: huge data/metadata, need for services & orchestration, evolving standards, organizations share spatial data, little infrastructure/expertise needed, resource pooling, managed services.
- Cloud advantages: on-demand scalability, better utilization, minimize IT management, focus on business, low start-up cost (**reduce CAPEX**), consumption billing, economy of scale, **green computing**.
- **Cloud actors**: **CSP/Broker, Customer, Negotiator (optional), SLA manager/Security auditor** (absent in current clouds).
- Cloud as provider: collection of **enterprise GIS (eGIS)** instances with **Resource service, Data service, Interface service**.
- Geospatial cloud model: **web services** are key; integrate heterogeneous back-end data services (inside/outside cloud); **PaaS** makes load balancing, replication, scaling transparent; data via **WFS, WMS** services; metrics: computation power, network bandwidth.
- Case studies: **merged road network** (highway + local roads) → shortest path; **merged water network** (canal + river) → buffer.
- Challenges: spatial DB implementation & scaling, **multi-tenancy**, policy management among tenants, geographically situated backups, data security.
- **Interoperability levels**: **data level**, **service level**, **security level**.
- Security concerns: **multi-tenancy** and **lack of control**. Questions to ask: which assets to deploy, value of assets, how they can be compromised.

WEEK 8

Docker Containers, Green Cloud, Sensor Cloud, IoT Cloud, Course Summary

8.1 Docker Containers

Basics

- **Docker** = container management service (initial release **March 2013**); motto **"develop, ship and run anywhere"**; supports Agile; solves **"works on my machine"** problem.
- Goal: **interoperability** – like shipping containers, package an app once and run anywhere (any physical/virtual machine, cloud).
- **Features**: smaller OS footprint via containers; easy cooperation among Dev/QA/Ops; deploy anywhere; **lightweight** → quick scale up/down; application-level virtualization; build once, run anywhere.

Components

- **Docker for Mac / Linux / Windows**, **Docker Engine** (builds images, creates containers), **Docker Hub** (registry hosting images), **Docker Compose** (define multi-container apps).

VM vs Container

- **Traditional virtualization**: Server → Host OS → **Hypervisor** (VMware, Hyper-V) → **Guest OS per VM** → apps. Resource-intensive, large images.
- **Docker**: Server → Host OS → **Docker Engine** → apps run as containers.
- **Containers** = abstraction at the **app layer**; package code + dependencies; **share the host OS kernel**; run as **isolated processes in user space**; images typically **tens of MB**; **start almost instantly**.
- **VMs** run a full guest OS each → heavier; **containers and VMs can be used together**.
- **Image** = lightweight, stand-alone, executable package (code, runtime, libraries, env variables, config). **Container** = **runtime instance of an image**; isolated by default.

Docker commands (terminology)

- **Image**: `images` (list local images), `run` (create container from image and execute), `tag`, `pull` (download from repository), `rmi` (delete local image).
- **Container**: `ps` (running), `ps -a` (all incl. stopped), `top` (processes), `start`, `stop`, `pause`, `rm` (delete container), `commit` (create image from container).
- **Dockerfile** = build script to create images automatically; version-controllable (Git/SVN). **Docker Hub** = public repository of images; **automated builds** from Dockerfiles on GitHub.
- Usage: developers eliminate environment problems; operators get **better compute density**; enterprises build agile delivery pipelines (Linux & Windows).

8.2 Green Cloud

- **Green computing** = environmentally responsible use of computers; designing, manufacturing, using, disposing of devices to reduce environmental impact. Green cloud aims at **efficient utilization + minimized energy consumption and CO_2**.
- Cloud advantages: low upfront spend, pay-as-you-go, global workforce, streamline processes, lower capital costs.

Why needed

- **Gartner 2007: IT industry ≈ 2% of world's CO_2 emissions. US EPA 2007: data centres ≈ 1.5% of US power**, more than doubled since 2000, cost $4.5 billion.
- Data centres consume **10–100× more energy per sq ft** than typical office building; **energy ≈ 10% of OPEX** (can rise to 50%); cooling costs $2–5 million/yr.
- Typical DC energy split: **IT equipment ≈ 40%**, **cooling ≈ 45%**, **power distribution ≈ 15%**.
- Carbon emission of data centres worldwide exceeds that of Argentina or the Netherlands. **Amazon**: energy-related costs ≈ **42%** of data-centre budget (15-year amortization). Google/Microsoft/Yahoo build DCs in barren desert/cold regions near cheap power.
- Shift from pure **performance** to **energy efficiency** with high service levels.

DC architecture & energy models

- Past: **two-tier** (access + core). Present: **three-tier** (**access, aggregation, core**), scales to **> 10,000 servers**; high-speed 3-tier with **ECMP** load balancing & **100 GE (IEEE 802.3ba)**.
- **Server energy model**: P_peak vs P_fixed (memory, disks, I/O); **idle server consumes ≈ 66% of peak power**; dynamic part depends on CPU frequency/load (DVFS).
- **Switch energy**: chassis ≈ 36%, line cards ≈ 53%, port transceivers ≈ 11%.

Green cloud architecture

- **Green Broker** (leases cloud services, schedules apps; layers: analyze user requirements → compute cost & CO_2 → choose greenest/cheapest offer), **CO_2 emission calculator**, **green policies**, **Green Middleware**.
- **Power Usage Effectiveness (PUE)** = Total facility power / IT equipment power (lower, closer to 1, is better).
- Conclusion: **Carbon-Aware Green Cloud Framework**.

8.3 Sensor Cloud

- Motivation: growth of sensors (RFID, cameras, phones), real-time web data, **Big Data**; cloud lacks real-world sensing.
- **WSN** (wireless sensor network): small, low-power, low-cost nodes with sensing, processing, memory, communication; applications in environment, healthcare, defence, etc.
- **Limitations of sensor networks**: hard to scale, proprietary vendor-specific designs, data not shareable, limited compute/storage, fixed/specific applications.
- **Sensor Cloud** = infrastructure allowing pervasive computation using **sensors as interface between physical and cyber worlds**, data-compute clusters as backbone, Internet as medium; supports **complete sensor-data life cycle** (collect, access, process, visualize, archive, share, search).
- Motivating scenario: tourist's phone uses GPS, physiological sensors, speech, images, maps, recommendations.

Framework

- **Sensor-Cloud Proxy** – interface between sensor resources and cloud; exposes sensors as cloud services; indexing/discovery; manages sensing jobs & data.
- **Sensor Network Proxy** – connects sensor networks lacking direct cloud connection; collects data continuously or on demand; **enhances scalability**.
- Another use case: **traffic-flow sensors** (plus humidity/temperature sensors) giving local weather data.

Virtual sensors

- **Virtual sensor** = emulation of a physical sensor that obtains data from underlying physical sensors; gives customized view with **location & distribution transparency**; solves the problem that sensor hardware can't run many VMs.
- Configurations: **One-to-many** (one physical sensor → many virtual sensors; middleware sets sampling rate considering all users), **Many-to-one** (region's sensors/WSNs aggregated for one virtual sensor), **Many-to-many** (combination), **Derived** (generalization; combines different physical sensor types, e.g., "is the wildlife habitat safe?").
- Sensor-cloud infrastructure: sensor **system management** + sensor **data management**; users create **virtual sensor groups** dynamically from templates.

8.4 IoT Cloud

- **IoT** = extending Internet to connect/communicate between devices and physical objects ("things"), each with **unique identifiers** and ability to transfer data; "any time, any place, anyone, anything".
- Basic IoT architecture – 3 building blocks: **Things, Gateway, Network and Cloud**.
- IoT aspects/challenges: **scalability, big data, real time, highly distributed, security, privacy, safety, speed**.
- Cloud complements IoT: pay-as-used on-demand, offload storage & processing, anywhere access.
- **iCOMOT** – toolset for managing IoT Cloud systems (top: IoT applications across IoT & cloud; middle: IoT cloud software layer; bottom: monitoring/control/configuration tools).
- Case study: **IoT-based vehicular data cloud** – multilayered platform with services such as **intelligent parking cloud service** (vacancy detection by sensors) and vehicle-tracking service.

8.5 Course Summary & Research Areas

- Course topics: intro & NIST model, properties/characteristics/disadvantages, architecture (stack, XaaS, deployment), service management (**SLA, economics**), resource management, data management (**GFS, HDFS, MapReduce**), security (IAM, access control, trust/reputation/risk), open-source & commercial clouds, research trends (**Fog, Sensor Cloud, Containers, Green Cloud**).
- Research areas: infrastructure & services (Storage-/Network-/Information-aaS), management (**composition, orchestration, federation, bridging, bursting, migration**, hybrid integration, green/energy mgmt, workload profiling, metering/monitoring/auditing), **security** (data privacy, access control, identity mgmt, **side-channel attacks**, Security-aaS), performance/scalability/reliability (**micro-services**), systems software/hardware (virtualization, provisioning orchestration), data analytics, service management (discovery, composition, QoS, semantic services), cloud + other technologies (**Fog, IoT Cloud, Containers**).

WEEK 9

Cloud–Fog–Edge Paradigm, Resource Management & Service Placement, Cloud Federation

9.1 Cloud–Fog–Edge Overview (Lecture 41)

- Cloud = "**Anything-as-a-Service**". **Fog computing**: data, processing, applications concentrated in devices **at network edge**; term introduced by **Cisco** to ease wireless data transfer to distributed devices in **IoT**; vision: apps on billions of devices run directly at the edge.
- Cloud-Fog-Edge: intelligence brought close to end-user; **base stations, routers, Wi-Fi gateways** can run applications; **end devices (sensors) do basic processing**; processing near devices → **lower response time → real-time apps**.

Issues at each layer

- **Cloud**: **latency**, **bandwidth** requirement, not designed for **volume, variety, velocity** of IoT data.
- **IoT/Edge devices**: limited **processing, storage, power**.
- **Fog layer benefits**: much lower latency (real-time apps), less network congestion, reduced cloud execution cost, better data location awareness, better handling of huge data.

Case study: Health Cloud–Fog–Edge

- 5-level hierarchy: **Level 0 Cloud → Level 1 ISP → Level 2 Area Gateway → Level 3 Mobile → Level 4 IoT devices**.
- Simulated using **iFogSim** (toolkit for modeling & simulating resource management in IoT/Edge/Fog; Gupta, Dastjerdi, **Ghosh**, Buyya 2017).
- Findings: **Network usage** very low for fog (only few positive cases go to **Confirmatory module in cloud**) vs high for cloud-only; **execution cost** higher in cloud-only (only cloud resources incur cost; other resources owned by the organization); **latency** of control loop **fixed/low in fog** (modules at Area Gateway) vs higher in cloud (modules in datacenter); energy compared for DC, mobile, edge.
- Prototype: **Raspberry Pi as fog devices + AWS as cloud**; customize resource allocation policy (devices, module requirements, **module placement policy**).

9.2 Resource Management in Cloud–Fog–Edge (Lectures 42–43)

- Cloud-only is not efficient for **time-sensitive** IoT applications; fog/edge process data **close to the source**, reducing bandwidth.
- **Fog and edge are not substitutes for cloud** – the three work together for improved latency, reliability, fault tolerance.
- **Three-layer model**: **client (edge) layer, fog layer, cloud layer**. If fog has no/limited resources, request **passes to cloud**. Components: **fog server manager** (assigns processors), VMs, etc.
- Trend: **decentralize** cloud resources toward edge – **micro data centers** or Internet nodes (routers, gateways, switches) augmented with compute.

Survey taxonomy (Hong & Varghese, ACM CSUR 2019)

- **Architectures** classified by **data flow, control, tenancy**.
  - Data flow = direction of workload/data movement (user→edge, cloud→edge). Control = centralized vs distributed. Tenancy = single vs multi-tenant.
- **Infrastructure**: **Hardware** (gateways, Wi-Fi APs, set-top boxes, small home servers, edge ISP servers, cars, drones, desktops/laptops/phones), **System software** (runs directly on fog hardware; needs **multi-tenancy & isolation**; **system virtualization** and **network virtualization**), **Middleware** (monitoring, coordination, orchestration, communication, protocols).
- **Algorithms (4 major)**: **Discovery** (identify edge resources), **Benchmarking** (capture performance of CPU/storage/network), **Load balancing** (priorities, fairness), **Placement** (find appropriate resources for deploying workloads).

Service (application) placement problem

- Fog infrastructure = **IoT devices (end layer), fog nodes, ≥1 cloud datacenter (cloud layer)**; fog is a highly virtualized platform between users and cloud.
- **Application placement** = mapping application components & links onto infrastructure graph (devices + physical links) satisfying requirements/constraints while optimizing an objective.
- **Constraints**: **resource** (CPU, RAM, storage, bandwidth), **network** (latency, bandwidth), **application** (e.g., **locality requirement**, dependencies/QoS).
- **Optimization objectives/metrics**: **latency, resource utilization, cost, energy consumption**.

- **Offloading** = moving server/application/data to the network edge; **user device → edge** (usually single hop; application partitioning), and **cloud → edge** (bring services closer to source).

9.3 Cloud Federation (Lecture 44)

- **Federated cloud** = deployment & management of **multiple external and internal cloud services** to match business needs; **union of several smaller parts performing a common action**.
- Collaboration among **CSPs** for capacity utilization, **interoperability**, catalog of services, insight into providers and SLAs.
- Motivation/benefits: maximize resource utilization, **minimize power consumption**, load balancing, global utility, expand CSP's global footprint; overcome service interruptions, lack of interoperability, degradation of services.
- Characteristics: inter-cloud organization with **voluntary** nature; maximum geographical separation; well-defined marketing system and **regulated federal agreement**.

Coupling levels

- **Loosely coupled**: limited interoperation; e.g., private cloud using external commercial cloud; **little/no control over remote resources** (no VM placement decisions), limited monitoring (CPU/memory/disk only), **no cross-site networks**.
- **Partially coupled**: **contract/framework agreement** between partner clouds; some control (e.g., **affinity rules**), more detailed monitoring.
- **Tightly coupled**: governed by **same cloud administration**; advanced control (exact VM placement), full monitoring, **cross-site networks, cross-site VM migration, high availability techniques**.

Federation architectures (Moreno-Vozmediano, Montero, Llorente 2012)

- **Hybrid / Bursting** – on-premise (private) cloud + public cloud resources for **peak demand**; **loosely coupled**.
- **Broker** – broker serves users and accesses several public clouds; **most common federation scenario**; advanced broker schedules by cost/performance/energy.
- **Aggregated** – two or more **partner clouds** aggregate resources into a larger virtual infrastructure; usually **partially coupled**.
- **Multitier** – multiple cloud sites (usually same corporation) each with its own cloud OS, managed by a **third (root/top) cloud OS** in a hierarchy; **tightly coupled**; exposes all sites as single cloud.
- Coupling order: **Bursting/Broker (loose) → Aggregated (partial) → Multitier (tight)**.

WEEK 10

VM Migration (Live Migration) & Containers (Docker, Kubernetes)

10.1 VM Migration – Basics

- **VM migration** = moving a running application/VM from one physical host to another; **processor state, storage, memory and network connection** move together.
- **Why migrate**: **load balancing**, **maintenance** (transparent), **reduce operational parameters/power** (consolidate VMs on fewer servers; put idle servers in low-power mode), **QoS/SLA violation** avoidance, **fault tolerance** (proactively handle failures), power management.
- **When**: remove a machine from service; relieve load on congested hosts.

Types

- **Cold / non-live migration**: VM **turned off or suspended** during migration → **downtime = total migration time**; bad for large VMs/real-time apps.
- **Hot / live migration**: VM **keeps providing service** during migration; entire VM (all user processes and kernel state) moved **without shutdown**. Two main approaches: **pre-copy** and **post-copy**.

Concerns

- **Minimize downtime** (time service is unavailable), **minimize total migration time**, avoid disrupting active services via **resource contention** (CPU, network bandwidth).

What to migrate

- **CPU context + main memory**; **disk** (not critical if **NAS** shared or disk mirrored); **network** (migrate IP address; advertise new MAC via **ARP reply**; switches learn new MAC location; both hosts on same LAN); **virtual I/O devices**.

Memory migration techniques

- **Push** – source keeps running while pages pushed; pages modified during transfer are **re-sent**.
- **Stop-and-copy** – source VM stopped, pages copied, new VM started (downtime).
- **Pull** – new VM runs; **page faults pull** missing pages from source.

Pre-copy (iterative) live migration

- **Phase 1 – Pre-copy (iterative push)**: several rounds; VM keeps running at source; **round 1 copies all pages**, later rounds copy **dirty pages** (modified during previous round).
- **Phase 2 – Termination**: stop when **any one** of: **(i) number of rounds exceeds threshold, (ii) total memory transmitted exceeds threshold, (iii) dirty pages of previous round fall below threshold**.
- **Phase 3 – Stop-and-copy**: VM **suspended** at source; remaining dirty pages + **CPU state** sent; VM **resumed at destination** (T_res = restart time).
- Drawback: frequently-dirtied pages sent many times → **long migration time**.

Post-copy live migration

- **Stop phase** (stop source, copy **CPU state**) → **Restart phase** (start destination VM with **empty memory**) → **On-demand copy** (pages fetched on page fault; techniques: **demand paging, active push, pre-paging, dynamic self-ballooning**).
- Advantages: each page transferred **once**; lower total migration time; some pages never needed.

Analysis formulas

- **Non-live**: **T_mig = V_m / R** (V_m = VM memory size, R = transmission rate); **T_down = T_mig**.
- **Pre-copy**: dirty ratio **r = (P × D) / R** (P = page size, D = dirtying rate, R = transmission rate); need **r < 1** for convergence.
  - Round 0: t_0 = V_m/R; round j: **t_j = r^j · V_m/R** (dirty data in round j = r^j V_m).
  - Total **T_mig = (V_m/R) · (1 + r + r² + … + rⁿ) + T_res = (V_m/R)·(1 − r^(n+1))/(1 − r) + T_res**.
  - **Downtime T_down = (V_m/R) · rⁿ + T_res** (only the last stop-and-copy round).
  - **Number of rounds**: stop when r^j·V_m < V_th ⇒ **n = min(⌈log_r(V_th/V_m)⌉, n_max)**.

Migrating multiple VMs

- **Serial migration** of m correlated VMs: first VM runs its pre-copy cycle while others keep running; when it enters stop-and-copy, **remaining (m−1) VMs are suspended** (to stop dirtying memory) and copied afterwards → **T_down = (V_m/R)·rⁿ + (m−1)(V_m/R) + T_res**; migration time ≈ m × single-VM time.
- **Parallel migration**: all m VMs start pre-copy simultaneously, **each gets R/m bandwidth**; effective dirty ratio becomes **m·r**; all enter/finish stop-and-copy together → **T_down = (m·V_m/R)·(m·r)^n(p) + T_res**; may need more rounds / higher downtime.

10.2 Containers

- **Container = lightweight virtualization**; packages **code + all dependencies** so app runs quickly/reliably across environments; **virtualizes the operating system**; abstracts app from environment (private DC, public cloud, laptop).
- Benefits: **agile development**, **separation of responsibility** (devs: app logic; ops: deployment), **workload portability** (Linux/Windows/Mac; VMs/bare metal; on-prem/public cloud), **resource efficiency & density**, consistency, scalability.
- A container is a **sandboxed process isolated from other processes**; **runnable instance of an image**; managed via API/CLI (create, start, stop, move, delete).

Traditional vs Virtualized vs Container deployment

- **Traditional**: apps on physical servers; **no resource boundaries** → allocation issues.
- **Virtualized**: multiple **VMs on one server**; isolation between VMs; better utilization & scalability; each VM has full OS.
- **Container**: **relaxed isolation – share the host OS kernel**; lightweight; own filesystem, CPU share, memory, process space; portable across clouds and OS distributions.
- VM = guest OS on top of host OS/hypervisor (heavy); container = far more lightweight.

10.3 Kubernetes

- **Kubernetes (K8s)** = portable, extensible, **open-source** platform for **managing containerized workloads and services** (declarative configuration + automation); name is Greek for **helmsman/pilot**. Operates at **container level**, not hardware level.
- Functions: **automating deployment, scaling, management** of containerized apps.
- **Cluster components**: **worker machines = nodes** run containerized apps; **Pods** = components of app workload hosted on nodes; **control plane** manages nodes and Pods (in production runs on multiple computers for fault tolerance & high availability); a cluster has **at least one worker node**.

10.4 Docker

- Platform to **"build, ship, and run any app, anywhere"**; standard way to solve **deployment**; docker analogy: dock workers loading standardized shipping containers.
- **Docker Engine** properties: **Standard** (industry standard, portable), **Lightweight** (shares host kernel; no OS per app → higher server efficiency, lower licensing cost), **Secure** (strong default isolation).
- **Docker image** = lightweight, standalone, executable package (code, runtime, system tools, libraries, settings); **images become containers at runtime** on Docker Engine; works for Linux and Windows apps.

Benefits / uses (Docker in Practice)

- Configuration effort separated from resource management; deployment is trivial (pull image & run) – runs on Red Hat, Ubuntu, CentOS as long as Docker is installed.
- Uses: **replacing VMs**, **prototyping software**, **packaging software**, **enabling microservices architecture**, **modeling networks** (hundreds/thousands of isolated containers on one machine), **full-stack productivity offline**, **reducing debugging overhead**, **documenting software dependencies**, **enabling continuous delivery (CD)**.

Architecture

- Docker on a host = **daemon (server with RESTful API) + client (CLI)**; client talks to daemon over **HTTP**; daemon manages images and containers, fetches images from a **registry** (public Docker Hub or **private registry** on internal network).

Demo (Lectures 49–50)

- Objective: run **MySQL + phpMyAdmin** on Docker. **MySQL** = widely used open-source relational DB; **phpMyAdmin** = web-based GUI to manage MySQL.
- **Without containers**: install MySQL, Apache web server, PHP, phpMyAdmin separately; moving to another machine needs reinstall + **backup & restore** of data. **With containers**: pull images and run → portable, quick.

WEEK 11

Dew Computing, Serverless Computing (FaaS), Sustainable Cloud Computing

11.1 Dew Computing (Lecture 51)

Concept

- **Dew computing (DC)** = paradigm combining core cloud-computing concepts with the **capabilities of end devices** (PCs, phones); aims to solve cloud's **reliance on Internet access** (works in **offline mode**).
- Cloud "family" hierarchy by distance from ground: **Cloud (far) → Fog (closer) → Edge → Dew (closest, on the end device)**. "Dew" = water droplets on the ground.
- DC is **user-centric, personalized**; located very close to end devices and is **first in the IoT–fog–cloud continuum**; **micro-service-based, vertically distributed hierarchy**; includes smart phones, watches, tablets at network edge.
- Primary goal: access a pool of raw data with **metadata** that can be created/edited/stored/deleted with minimal network management (offline).
- **Typical example: Dropbox** – keeps local copies + cloud copy; works offline, **synchronizes** when connection returns.

Key features

- **Independence** – local device must provide service **without continuous Internet connection**.
- **Collaboration** – application must **connect to the cloud and synchronize** data when appropriate.

Dew service models (applications)

- **Infrastructure-as-Dew (IaD)** – local device dynamically supported by cloud services.
- **Software-in-Dew (SiD)** – **configuration and ownership of software saved in cloud** (e.g., Apple App Store, Google Play).
- **Platform-in-Dew (PiD)** – software development suite installed locally with settings/app data **synchronized to cloud**.
- **Storage-in-Dew (StiD)** – local storage partially/fully **copied into cloud** (e.g., Dropbox).
- **Web-in-Dew (WiD)** – local device holds a **duplicated fraction of the World Wide Web**.

Architecture

- **Dew Virtual Machine (DVM)** = isolated environment on local machine to execute the **dew server**; components: **Dew Client Program, Dew Client Service, Dew Server, Dew DBMS**; **Single-Super-Hybrid-Peer (P2P) communication link** to cloud server; IoT devices & sensors connect to host machine.
- **Dew server functions**: serve requested services; **synchronize local and remote data**. Goals: **Data replication, data distribution, synchronization**.

Challenges

- **Power management, processor utility, data storage, viability of OS, programming principles, database security.**

11.2 Serverless Computing (Lectures 52–53)

Basics

- **Serverless computing** = providing **backend services on an as-used basis**; developers write/deploy code without worrying about infrastructure; eliminates system administration; **servers still exist** but are invisible.
- Developers write **functions**, provider runs them in **shared runtime environment**, scaling automatically (many functions in parallel); **pay only when code runs**.
- Serverless target = **programmers**; classic cloud target = **system administrators**. Event-driven, granular applications.
- Code runs in **almost-stateless, event-triggered, ephemeral containers** (may last for one invocation); containerization → faster boot ("cold start" reduction).
- Two categories: **Backend-as-a-Service (BaaS)** and **Function-as-a-Service (FaaS)**.
- **BaaS**: replace server-side components with **off-the-shelf services** (authentication, database, cloud storage, hosting); e.g., **Google Firebase**; logic mostly in frontend.
- **FaaS**: cloud provider manages **resources, lifecycle and event-driven execution** of **small, stateless functions**; each function = small part of the app; short execution time; event-triggered.

Challenges

- **Asynchronous calls** add complexity; **functions calling functions** (complex debugging, loose isolation, double billing for synchronous calls); **too many libraries** (image size limit, longer warm-up); **too many technologies** (maintenance complexity); **too many functions** (poor reuse; inactive functions); cold start; statelessness.

Major providers & platforms

- **AWS Lambda** (Amazon; event-driven; code runs in response to events such as **S3 file add/delete, API Gateway HTTP request**; languages: Java, Python, Go, C#, etc.; **billed only for compute time while code runs**).
  - Concepts: **Function**, **Runtime**, **Event source** (AWS service that triggers the function), **Layers**, **Log streams**.
  - Workflow: upload code → configure trigger/event details → Lambda executes on trigger → charges only for execution.
- **Google Cloud Functions** – serverless execution environment of **single-purpose functions** attached to events from cloud services (Cloud Storage, Pub/Sub, Firebase, Stackdriver Logging, Firestore, Compute Engine, BigQuery, direct **HTTP**).
  - **Event** (occurs in GCP), **Trigger** (event + response binding), **Event data** (passed to function). Multiple event occurrences → multiple invocations; function should be single-purpose, short, and signal completion.
- **Azure Functions** – serverless solution: write less code, maintain less infrastructure, save costs; languages **C#, Java, JavaScript, PowerShell, Python** (custom handlers); patterns: serverless APIs, event & stream processing, IoT data processing, ML pipelines, automation, integration/pub-sub; scenarios: retail web backend, financial mobile backend.

11.3 Sustainable Cloud Computing (Lectures 54–55)

Why

- CSPs depend on **Cloud Data Centers (CDCs)**; financial + **carbon-footprint** costs hurt sustainability. Focus: **minimize energy consumption & carbon footprint while ensuring reliability**.
- CDC energy use rising; projected **≈ 8000 TWh by 2030**. Under-/over-loading of resources wastes energy.
- Major CSPs (**Google, Amazon, Microsoft, IBM**) plan **renewable-powered** data centers; relocate data centres based on **waste-heat recovery, access to green energy, free cooling**.
- Need **holistic, energy-aware resource management** (servers, storage, memory, network, cooling).

Conceptual model (Buyya & Gill, 2018) – layered architecture

- **Cloud architecture** (SaaS, PaaS, IaaS), **Cooling manager** (thermal alerts/temperature), **Power manager** (controls **renewable vs grid energy** using **Automatic Transfer Switch (ATS)** and **Power Distribution Unit (PDU)**), plus resource scheduling & management layers. Grid energy used for deadline-oriented workloads to keep reliability.

Reliability vs sustainability

- **Issues – energy**: reduce CDC energy, reduce under/over-loading (load balancing), minimize heat, reduce carbon footprint, improve bandwidth/compute/storage management. **Reliability**: detect failures, etc.
- **Trade-off**: replicating services for reliability **increases energy consumption**; **frequent server on/off (power modulation) reduces reliability** of components (storage, memory). Reducing energy improves utilization, reliability, performance.

Components of sustainable cloud computing

- **Application model** (data parallel, function parallel, message passing); **resources targeted in energy management** (processor, memory, storage, network, cooling); **thermal-aware scheduling** (architecture single-/multi-core; mechanism **reactive or proactive**; minimize cooling set-point temperature); **capacity planning** (power infrastructure, IT resources, workloads; ROI); **waste heat utilization** (**vapor-absorption-based cooling/free cooling** can make **PUE ideal**).

Taxonomy (Gill & Buyya, ACM CSUR 2018)

- Research categories: **application design, sustainability metrics, capacity planning, energy management, virtualization, thermal-aware scheduling, cooling management, renewable energy, waste heat utilization**.
- **Virtualization** supports energy-efficient **VM migration, elasticity, load balancing, consolidation, fault tolerance, scheduling**.
- **Renewable energy** (solar, wind, water) gives near-zero CO_2.
- Next-generation cloud must be **energy efficient and sustainable**, powered by renewable energy.

WEEK 12

Cloud in 5G Era, Cyber-Physical Systems, Spatial Cloud, Internet of Health Things (IoHT) Case Study

12.1 Cloud Computing in the 5G Era (Lecture 56)

5G basics

- **5G = fifth-generation mobile network**; global wireless standard after 1G–4G; connects people, machines, objects, devices; delivers **multi-Gbps peak data rates, ultra-low latency, higher reliability, massive capacity, uniform user experience**.
- Generations: **1G (1980s) analog voice**; **2G (early 1990s) digital voice (CDMA)**; **3G (early 2000s) mobile data**; **4G LTE (2010s) mobile broadband**; **5G** unified, forward-compatible platform.

Three 5G service categories

- **eMBB – Enhanced Mobile Broadband**: indoor/outdoor broadband, enterprise collaboration, **AR/VR**.
- **mMTC – Massive Machine-Type Communications**: **IoT**, asset tracking, smart agriculture, smart cities, energy monitoring, smart home.
- **URLLC – Ultra-Reliable Low-Latency Communications**: **autonomous vehicles, smart grids, remote patient monitoring/telehealth, industrial automation** (mission-critical).

5G and cloud

- 5G is the "perfect companion" to cloud (distributed, diverse compute/storage); on-premises and **edge data centers** bridge gap between low-latency devices and distant clouds → **heterogeneous, distributed computing**; need **end-to-end orchestration**.
- Two-way relationship: (1) cloud must evolve to meet 5G needs → **edge, mobile edge, fog computing**; (2) 5G is being **"cloudified"** through **network softwarization, NFV (Network Function Virtualization), SDN**.
- **Edge computing in 5G**: brings cloud capabilities near users/UEs to overcome **high latency**; 5G must handle **massive data** from mobile/IoT, **stringent QoS (ultra-low latency, high throughput)**, **heterogeneity/interoperability**.
- Edge applications: healthcare, entertainment & multimedia, **VR/AR/MR**, **tactile Internet**, IoT, factories of the future, emergency response, **Intelligent Transportation Systems (ITS)**.
- **5G + Mobile Cloud Computing**: resource sharing, improved reliability (backup in cloud), **offload processing** to cloud → fewer device resources used.

12.2 Cyber-Physical Systems (CPS) & Cloud (Lecture 57)

- **CPS = orchestration of computers and physical systems**; embedded computers **monitor and control physical processes via feedback loops**; physical processes affect computations and vice versa.
- Term **coined around 2006 by Helen Gill (NSF, USA)**. CPS is about the **intersection, not the union**, of physical and cyber.
- CPS = "**integration of computation with physical processes**" using **sensors and actuators**; "computing as a physical act". Interlinked networks of sensors, actuators and processing devices → **Smart Networked Systems and Societies (SNSS)**.
- Goals: improve **adaptability, autonomy, efficiency, functionality, reliability, safety, usability**.
- **Cyber-Physical Cloud Computing (CPCC)** (NIST report, 2013): "a system environment that can **rapidly build, modify and provision CPS** composed of a set of **cloud-based sensor, processing, control and data services**".
- **CPCC benefits**: efficient resource use, **modular composition**, rapid development & scalability, smart adaptation to environment at every scale, reliable & resilient architecture.
- Examples: **cloud-based CPS for intelligent monitoring of machining processes**; **Cloud-Edge computing framework for Cyber-Physical-Social services** (Wang, Yang et al., IEEE Commun. Mag. 2017).

12.3 Spatial Cloud Computing – Case Study I (Lecture 58)

- **Spatial/geospatial data** = information describing objects/events with a **location on/near Earth's surface**; combines **location + attribute + temporal** information.
- **Spatial analysis** = solving location-oriented problems; goes **beyond mapping** to study characteristics of places and relationships; supports decision making.
- **Spatial analytics + cloud**: elastic, on-demand platform integrating observation systems, parameter-extraction algorithms, simulations, visualization & decision support; helps search/access geospatial data and configure compute for intensive simulation models.

- **Spatial cloud**: shared **resource pooling**, choice of deployment/service/business models, managed services prevent data loss.
- **Advantages**: **easy to use** (deploy infrastructure with a click/API), **scalability**, **cost** (usage-based), **reliability** (enterprise hardware; multiple clouds), **risk** (change instantly, even OS).
- Typical architecture: organizations share spatial data; easy access to spatial services; integrate latest databases, merge disparate systems, exchange information.

Mobility analytics & Traj-Cloud

- **Mobility/trajectory analytics** on cloud for **smart-city / ITS** (location-based services, transport planning, urban dynamics).
- **Traj-Cloud** (Ghosh & Ghosh, COMSNETS 2019) services on **Google Cloud Platform (GCP)**:
  - **TS1 Trajectory data indexing** – input GPS trace + semantic info; output **spatio-temporal indices**; uses **Google BigQuery and Cloud SQL**.
  - **TS2 Trajectory map-matching** – project GPS trace onto road network using **MapReduce-based platform**.
  - Further services for semantic analysis of movement behaviour.

12.4 Internet of Health Things (IoHT) – Case Study II (Lectures 59–60)

Cloud–Fog–Edge–IoHT framework

- **Fog computing** brings cloud closer to data-producing sensors; routers/servers/switches act as **fog nodes**; used for real-time apps; **health data processed at edge/fog to reduce latency, network usage and cloud cost**.
- Cloud limitations: **latency, large data volume, bandwidth**. IoT device limitations: **processing, storage, power**. Fog–edge benefits: reduced latency (real-time), less congestion, lower cloud execution cost, better handling of colossal sensor data, **location awareness**.
- **Objectives**: design **fog-edge health model** to reduce latency, network usage, cloud cost; **test in iFogSim**; develop **customized wearable device**; implement on hardware; study **dew computing** in the health scenario.
- Overall workflow: conceptualization/modelling → **iFogSim simulation** & evaluation → **hardware implementation** (custom body sensor) → **activity detection using accelerometer** and **heart-attack prediction** → **dew-based** model.
- **Hierarchical topology**: **Cloud (Level 0) → Fog (Level 1) → Edge (Level 2) → IoT (Level 3)** (in the 5-level version: Cloud, ISP, Area Gateway, Mobile, IoT).
- **Device configuration (typical)**: Cloud MIPS 44800, RAM 40000 MB; ISP/AreaGW 2800 MIPS, 4000 MB; Mobile 350 MIPS, 1000 MB. **Link latencies (ms)**: body sensor→mobile 1, mobile→Area GW 2, Area GW→ISP GW 2, **ISP GW→Cloud 100**, mobile→display 1.

Application placement (modules)

- **Fog-based model**: Client module → Mobile (edge); **Data filtering, Data processing, Event handler → Area Gateway (fog)**; **Confirmatory module → Cloud**.
- **Cloud-based model**: Client module at mobile; **all other modules in Cloud**.
- Simulation configs: 1–5 with **1, 2, 4, 8, 16 Area Gateways** and **4, 8, 16, 32, 64 users**.

Performance results (fog vs cloud)

- **Latency**: fog latency **fixed/low** (control-loop modules at area gateway); cloud higher and grows.
- **Network usage**: fog **very low** (only positive cases reach cloud Confirmatory module); cloud high.
- **Cost of execution**: fog lower (only cloud resources incur cost; others owned by org); cloud higher.
- **Energy**: mobile energy same in both; fog/DC energy changes with configuration.

Hardware implementation

- **Customized body sensor** (BP & pulse meter), **NodeMCU ESP8266** (Arduino-like board with Wi-Fi), **ADXL345 accelerometer** (x, y, z), **Raspberry Pi 3 as fog device** (64-bit, 1 GB RAM, Wi-Fi, Bluetooth), **AWS as cloud**.

Activity detection (accelerometer)

- Pipeline: data extraction → **$A = \sqrt{x^2 + y^2 + z^2}$** → **5-point smoothing** (average of 2 preceding, itself, 2 succeeding) → **feature extraction** (max, min, mean amplitude, std deviation, energy in time domain, energy in frequency domain) → **min-max normalization $Y = (x - \min)/(\max - \min)$** → **KNN classifier, K = 3** (chosen by 5-fold cross-validation using GridSearchCV).

Cardiac attack prediction (illustrative logic, no clinical significance)

- Uses KNN activity factor f+1 to scale: p = BPM/f, s = systolic/f, d = diastolic/f. **Alarm if $p \geq 170$ AND $s \geq 180$ AND $d \geq 120$** (thresholds BPM ≥ 170, diastolic ≥ 120, systolic ≥ 180).

Dew-based extension (CONFRONT framework)

- **Dew computing** = on-premises software-hardware organization paradigm where the on-premises computer provides functionality **independent of cloud services yet collaborative** with them.

- Workflow: sensor collects data → if **cloud connectivity** available → **fog/cloud processing**; else if **dew connectivity** available → **dew-based processing**, **sync with cloud** later; else wait.
- **Comparison (Cloud vs Cloud-Fog-Edge vs With Dew)**: on-premise resource utilization **Low / Sub-optimal / Optimal**; connectivity **Internet / Local / Local**; uptime **Low / High / High**; bandwidth **High / Low / Low**; latency **High / Low / Low**; infrastructure requirement **Low / Moderate / Moderate**; processing power **High / Limited / Limited**; data storage **High / Low / Moderate**.
- Reference: **CONFRONT – Cloud-fog-dew based monitoring framework for COVID-19 management** (Poonia, Ghosh, …, Buyya; *Internet of Things*, Elsevier 2021).